V0.1 · NMOS 6502 · 320×200 · 4 VOICES

DEVELOPER'S GUIDE

MACHINE REFERENCE & PROGRAMMING MANUAL

SYSTEM VERSION 1.0

Fanticon Developer's Guide

The complete reference for programming the Fanticon system — the console, its tools, and every register in between.

Covers the Fanticon hardware, the native editor and command console, the macro assembler, the graphics and music editors, and the complete memory map, opcode, and file-format references, organized as a path from first power-on to a finished cartridge.

TABLE OF CONTENTS

Contents

PART IV — SOUND

PART V — INPUT, INTERRUPTS & TIMING

PART VI — SHIPPING A GAME

PART VII — GAME DEVELOPMENT PATTERNS

REFERENCE APPENDICES

CART 01

01

WELCOME TO THE CONSOLE

What Fanticon is, why it is built this way, and how to get the host running on your machine.

CH 0 WHAT IS FANTICON · CH 1 INSTALL & RUN THE HOST

CHAPTER 0 · PART I

What Is Fanticon?

> Fanticon is a fantasy console: an imaginary late-1980s home computer, emulated down to the cycle. There was no 1988 Fanticon on store shelves, but every register, every cycle, and every scanline in this machine behaves as documented here. You program it the way developers programmed 8-bit hardware of that era: directly, in 6502 machine code, against memory-mapped chips, with no operating system standing between your program and the metal.

That is the whole idea. Modern engines hide the machine from you. Fanticon hands it back. Learning Fanticon means learning the NMOS 6502 CPU, a tile-and-sprite graphics architecture in the tradition of the NES and similar consoles of its era, and a small PSG-style sound architecture. Skills you build here transfer directly to retro game programming, emulator development, and low-level systems work in general.

HARDWARE SNAPSHOT

NMOS 6502 CPU at 3.144 MHz · 320×200 indexed-color video at 60 Hz · 256-color palette (16 banks of 16) · 32 hardware sprites · two pulse, one triangle, and one noise audio voice · two controllers · two interval timers · up to 4 MiB of banked cartridge ROM · up to 64 KiB of battery-backed save RAM.

Why a fantasy console, and why this one

Fantasy consoles exist to make retro-style game programming approachable without requiring a shelf of forty-year-old hardware or the quirks of one specific dead platform. Fanticon leans further into authenticity than most: rather than offering a friendly high-level scripting API, it puts you directly on the bus with the same NMOS 6502 that powered the Apple II, Commodore 64, NES, and Atari machines of the era, undocumented opcodes and all, so code that depends on exact NMOS behavior works exactly as it would on original hardware.

Fanticon borrows its mental model from several historical machines rather than cloning any one of them:

- The 6502 and its memory-mapped I/O philosophy come from MOS Technology's own documentation for the 6500 family.
- Simple timers and shared interrupt flags follow the precedent set by the 6522 VIA, a companion chip found in countless 6502-based machines.

- The display borrows the programmable-raster philosophy of controllers like the MC6845, combined with game-oriented tile and sprite fetchers.
- The audio processing unit has the recognizable two-pulse, triangle, and noise shape of the Nintendo Entertainment System, with a simpler, more direct register model.

Six design principles

Every part of how Fanticon's hardware is put together follows from six principles baked into the system architecture:

Principle	What it means in practice
Small enough to memorize	The entire I/O map fits on one page. Every standard device lives inside the 256-byte `$C000` page.
Hardware-shaped games	Tiles, sprites, palette banks, scanline timing, interrupts, and bank switching are things your code actively manages — not details an engine papers over.
No accidental bottlenecks	The CPU never has to redraw 64,000 pixels just to move a character. Tile and sprite hardware does that work.
Cycle-deterministic	A cartridge behaves identically on every host and can intentionally race the raster beam for effects like palette splits.
Friendly constraints	Direct registers replace historical interface rituals (like serial controller strobing) that added cost without adding creative value.
Room to grow	Reserved addresses read `$FF` and ignore writes until a future hardware revision assigns them, so software written today keeps working.

Fanticon is built around one tile-or-bitmap background layer plus 32 hardware sprites, a direct 6502-driven bus, and a fixed 60 Hz NTSC-rate timing model. Design around those constraints deliberately — they are the same kind of constraints that shaped a generation of 8-bit games, and Part VII of this book is dedicated to working with them by genre.

The rest of this book walks the whole path: finding your way around the console, writing and assembling your first program, drawing tiles and sprites, making sound, handling input and interrupts, and finally packaging, optimizing, and shipping a cartridge. Reference appendices at the back collect the complete opcode tables, memory map, and file formats for when you need exact answers fast.

CHAPTER 1 · PART I

Launching Fanticon & Finding Your Way Around

> This chapter assumes Fanticon is already installed on your machine. It is a quick orientation to the app you are about to live inside for the rest of this book — not a build guide.

Starting up

Open the Fanticon application. A resizable window appears at an 8:5 aspect ratio, the centered Fanticon logo displays for five seconds, and then Fanticon drops you into the native **Editor mode** command console — an 80×50 character-grid terminal that is your home base for everything in this book. The logo ignores keyboard and mouse input for its first 500 ms so an accidental keypress on launch cannot skip past it; after that, any key or click dismisses it early.

TWO MODES, ONE WINDOW

Editor mode runs native host tools (the console, text editor, graphics editor, music tracker, debugger) at 640×400. **Game mode** runs a cartridge through the emulated 320×200 machine. Editor tools never execute on the 6502 — they are ordinary native code, which keeps development responsive and keeps Game mode's behavior accurate to cartridge execution. Press `F1` for Editor mode and `F2` for Game mode at any time, or type `EDITOR` / `GAME` at the prompt.

Launching a cartridge directly

Most of the time you will build and launch a project from inside the editor with `RUN` (Chapter 5). Fanticon can also open one specific cartridge straight from your operating system's command line, bypassing the editor entirely:

```
fanticon-app /path/to/GAME.FCN
```

A cartridge launched this way ignores `Escape` — there is no editor to return to. A game launched from inside the editor with `RUN`, by contrast, returns to the editor when you press `Escape`, after flushing any battery-backed save RAM to disk. Use whichever entry point matches what you are doing: the direct launch for showing someone a finished game, `RUN` for everything you are actively building.

Your project folder

The console's root directory (`/` at the `/>` prompt) maps to a folder on your computer: your operating system's Documents folder, inside a `Fanticon` subdirectory. Fanticon creates it automatically the first time it runs and never lets console paths escape it — everything you build and save lives there in ordinary files you can also browse from outside Fanticon.

Fanticon ships with its example cartridges built in and mirrors them into that folder under `/demos` the first time it runs, leaving anything else you have already created elsewhere in the folder untouched. That gives you eight complete, working cartridges to read, run, and take apart from day one.

TRY IT: TOUR THE DEMOS

Once the console appears, type `DIR /demos` to list the eight example cartridges installed with Fanticon: `audio`, `bitmap`, `graphics`, `music`, `raster`, `sprites`, `tiles`, and `wave`. Later chapters walk through every one of them. Change into any folder and type `RUN` to build and launch it on the spot.

With Fanticon running and the demo cartridges in view, you are ready to meet the console from the inside: the command prompt itself, in Chapter 2.

CART 02

02

YOUR FIRST CARTRIDGE

The command console, the 6502 in brief, your first assembled program, a running cartridge, and the debugger that watches it think.

CH 2 CONSOLE · CH 3 THE 6502 · CH 4 HELLO FANTICON · CH 5 FIRST CARTRIDGE · CH 6 DEBUGGER

CHAPTER 2 · PART II

The Command Console

> Everything in Fanticon starts at the prompt. The console is a native tool — it never runs on the emulated 6502 — so it stays fast and responsive no matter what your cartridge is doing.

The filesystem

Console paths are lowercase and accept either `/` or `\` as a separator. Every file and directory name follows the classic 8.3 rule: 1–8 characters, an optional dot, and a 1–3 character extension, using letters, digits, `_`, and `-` only. New names are created lowercase; an existing mixed-case folder such as `MyGame` is matched case-insensitively as `mygame`. Absolute paths begin at Fanticon's `/`, and `..` can never climb above it.

Command	Effect
`HELP`	List current commands
`CLS` / `CLEAR`	Clear the terminal
`MODE`	Print the active application mode
`EDITOR` / `GAME`	Switch application mode
`ECHO text`	Print text
`VERSION`	Print the Fanticon host version
`COLOR [BG FG]`	Show or set the console's background/foreground palette indexes
`EDIT [file]`	Open the text editor
`CD [path]`	Change directory, or print the current one
`MKDIR` / `RMDIR path`	Create / remove an empty directory
`RM` / `DEL file`	Remove one file
`DIR [path]` / `LS [path]`	List a directory
`ASM source [output]` / `BUILD`	Assemble source, or build the current project
`RUN [file.fcn]`	Build & launch the project, or launch an image
`DUMP file [offset [length]]`	Hex/ASCII dump of a file

Unknown commands print a visible error and return you to the prompt — nothing is silent. Backspace edits the current line, Enter submits it, and output scrolls once it reaches the bottom row.

TRY IT: DUMP

`DUMP` shows any file eight bytes per row, sized to the 40-column display, with addresses, byte values, and printable ASCII in distinct colors. Zero-filled regions are dimmed so they visually recede. Try `DUMP /demos/sprites/MAIN.ASM` or, once you have built something, `DUMP HELLO.BIN $80 64` to inspect 64 bytes starting at offset `$80`. Numeric arguments accept decimal, `$hex`, or `0xhex`.

The text editor

Run `EDIT` for a blank document or `EDIT NOTES.TXT` to load a file from the current directory. It is Fanticon's own tool, drawn with the same character display as the console — not your operating system's native text box.

The menu bar has six menus, opened with `F10`/`Alt` or their own hotkey: **File** (`Alt+F`), **Edit** (`Alt+E`), **Build** (`Alt+B`), **Debug** (`Alt+D`), **Music** (`Alt+M`), and **Help** (`Alt+H`). On macOS, `Option+F`/`Option+E` work the same way and `Cmd` doubles for `Ctrl` shortcuts.

Shortcut	Action
`Ctrl/Cmd+N` / `O` / `S`	New / Open / Save
`F2` / `F3`	Save / Open shortcut
`Ctrl/Cmd+A/C/X/V/Z`	Select All / Copy / Cut / Paste / Undo
`F5` or `Ctrl/Cmd+B`	Assemble the current buffer
`F4` / `Shift+F4`	Next / previous build diagnostic
`Shift+F5`	Build & Run

Opening or saving a file ending in `.ASM` or `.INC` automatically switches on **6502 assembly mode**: a Merlin-inspired four-column layout (label, opcode, operand, comment, at fixed columns), Catppuccin Mocha syntax coloring, and Tab-to-next-field editing. This is the mode you will live in for the rest of the book.

TIP

In assembly mode, pressing Enter at the end of a line reformats it into the four-column layout immediately. Selecting text with Shift deliberately suppresses reformatting, so you can select and copy source without Fanticon rewriting it out from under you.

CHAPTER 3 · PART II

Meet the 6502

> This chapter is a working primer, not the complete reference — the full opcode and cycle tables, addressing-mode formulas, and undocumented-instruction catalog live in Appendix A for when you need exact answers.

Registers

Register	Width	Purpose
`A`	8 bits	Accumulator — arithmetic, logic, most data movement
`X`, `Y`	8 bits	Index registers — indexed addressing, loop counters
`SP`	8 bits	Stack pointer; the physical stack address is `$0100 + SP`
`PC`	16 bits	Address of the next instruction byte
`P`	8 bits	Status flags, written `NV-BDIZC`

The 6502 is an 8-bit, little-endian processor addressing 65,536 bytes, `$0000` through `$FFFF`. All arithmetic wraps: incrementing `$FF` yields `$00`. A 16-bit value stored at address `$2000` keeps its low byte at `$2000` and high byte at `$2001`.

Status flags (`NV-BDIZC`)

Bit	Flag	Set means…
7	`N` Negative	Result bit 7 is 1
6	`V` Overflow	Signed arithmetic overflowed
3	`D` Decimal	`ADC` / `SBC` use BCD adjustment
2	`I` Interrupt disable	Maskable IRQ is ignored
1	`Z` Zero	The result was zero
0	`C` Carry	Carry / no-borrow / shifted-out bit

For subtraction and comparison, `C=1` means **no borrow** — after `CMP value`, `C=1` means `A >= value` unsigned.

Zero page, the stack, and vectors

Every 6502 program leans on three fixed regions:

Address	Purpose
`$0000–$00FF`	Zero page — shorter, faster addressing
`$0100–$01FF`	Hardware stack
`$FFFA–$FFFB`	NMI vector
`$FFFC–$FFFD`	RESET vector
`$FFFE–$FFFF`	IRQ/BRK vector

Reset does *not* give you a known `A`, `X`, or `Y`. Every program should establish its own starting state explicitly:

```
RESET   SEI             ; mask IRQ while initializing
        CLD             ; use binary arithmetic
        LDX   #$FF
        TXS             ; begin with a known stack position
        ; ...initialize RAM and Fanticon devices here...
        CLI             ; enable interrupts only once handlers are ready
MAIN    JMP   MAIN
```

Addressing modes, briefly

Mode	Example	What it addresses
Immediate	`LDA #$40`	The literal byte `$40`
Zero page	`LDA $40`	Address `$0040`
Zero page,X	`LDA $40,X`	`($40+X) & $FF` — wraps in zero page
Absolute	`LDA $2040`	Address `$2040`
Absolute,X/Y	`LDA $2040,X`	`$2040 + X`
Indirect indexed	`LDA ($40),Y`	Read a pointer from zero page, then add Y — the classic buffer-walking mode
Indexed indirect	`LDA ($40,X)`	Read a pointer from `($40+X)&$FF` — selects among adjacent pointers

The instruction families

You do not need to memorize all 151 official mnemonics before writing your first program — most 6502 code leans on a small working set:

- **Move data:** `LDA/LDX/LDY`, `STA/STX/STY`, `TAX/TAY/TXA/TYA/TSX/TXS`

- **Do arithmetic:** `ADC`, `SBC`, `INC/DEC`, `INX/INY/DEX/DEY` — remember to `CLC`/`SEC` before a chain
- **Compare and branch:** `CMP/CPX/CPY`, then `BEQ/BNE/BCC/BCS/BMI/BPL/BVC/BVS`
- **Logic and shifts:** `AND/ORA/EOR`, `ASL/LSR/ROL/ROR`, and `BIT` for testing memory without touching `A`
- **Control flow:** `JMP`, `JSR`/`RTS` for subroutines, `BRK`/`RTI` for interrupts
- **Stack and flags:** `PHA/PLA`, `PHP/PLP`, `SEI/CLI`, `SEC/CLC`, `SED/CLD`

A CYCLE-ACCURATE CPU

Fanticon emulates exact NMOS bus timing, including page-crossing penalties, dummy reads before indexed stores, and the famous `JMP ($12FF)` page-wrap bug. This matters the moment you memory-map a device register: never use a read-modify-write instruction (like `INC` or `ASL`) on a hardware register unless its documentation explicitly allows the extra reads and writes that instruction performs. Chapters 7 onward call this out wherever it applies.

With registers, flags, addressing, and a safe reset routine in hand, you have everything needed to write and assemble code — which is exactly what Chapter 4 does.

CHAPTER 4 · PART II

Hello, Fanticon!

> Every Fanticon program starts life the same way: as source text assembled into raw bytes. This chapter builds your first one and gets comfortable with the assemble-fail-fix loop before Chapter 5 turns it into something that runs on screen.

Write the source

Type `EDIT HELLO.ASM` at the console. Assembly mode switches on automatically because of the `.ASM` extension. Enter this short program:

```
          ORG   $8000
START     LDX   #0
LOOP      LDA   MESSAGE,X
          BEQ   DONE
          INX
          BNE   LOOP
DONE      RTS
MESSAGE   ASC   "HELLO, FANTICON"
          DFB   0
          END
```

This does not draw anything yet — it is a raw subroutine that walks a null-terminated string in `X` until it hits the zero byte written by `DFB 0`. That is deliberate. The goal here is the assembler workflow itself: labels, `ORG`, string data, and a clean assemble.

Assemble it

Press `F5` (or `Ctrl/Cmd+B`). Fanticon shows a progress dialog, then a success dialog reporting the output filename, its origin address, and its byte count — `HELLO.BIN`, origin `$8000`, 21 bytes. The same thing happens from the console with:

```
ASM HELLO.ASM
```

By default the output file swaps the source extension for `.BIN`. Now look at the bytes you produced:

```
DUMP HELLO.BIN
```

You should see `A2 00 BD 09 80 F0 06 E8 D0 F7 60 48 45 4C 4C 4F...` — `LDX #0` as opcode `$A2`, immediate operand `$00`, then the absolute-indexed `LDA MESSAGE,X` as `$BD` with the resolved address of `MESSAGE`, and finally your ASCII string. The assembler turned your labels into addresses and your text into raw bytes — nothing more.

TRY IT: BREAK IT ON PURPOSE

Change `BEQ DONE` to `BEQ NOWHERE` (a label that does not exist) and assemble again. A build-error dialog appears with the exact diagnostic, the editor jumps to the offending line, and that line turns red. Press `F4` to jump between multiple errors. Fix the typo and the red tint clears the moment you edit the line. At the console, the same failure prints as `source:line:column message` and no new binary is written — a bad build never silently overwrites a good one.

Numbers, strings, and directives you just used

Directive	Purpose
`ORG expr`	Set the address for the bytes that follow
`name EQU expr`	Define a named constant
`DFB` / `DB` / `BYTE`	Emit bytes or strings
`DW` / `DA` / `WORD`	Emit 16-bit values, low byte first
`ASC` / `TEXT`	Emit a quoted ASCII string
`HEX`	Emit hex byte pairs directly
`DS expr`	Reserve and zero-fill a number of bytes
`PUT` / `USE` / `INCLUDE`	Assemble another file at this point
`END`	Mark the logical end of source

`$` or `0x` prefixes hexadecimal, `%` prefixes binary, and bare digits are decimal. `<` and `>` extract the low and high byte of an expression — you will use those constantly once you start loading 16-bit pointers. The full directive, expression, and modern macro reference is in Appendix C.

MACROS SCALE PAST COPY AND PASTE

Fanticon macros can declare named parameters with defaults, use private `@LOCAL` labels that are unique for every invocation, select source with `IF`, and generate repeated source with `REPEAT`. The bitmap, tile, wave, and audio demos use these features where register setup or table-shaped code would otherwise be duplicated. Start with `PMC NAME;arg1;arg2`; Appendix C gives the complete syntax.

You now have a working assemble loop. Chapter 5 wraps a program like this one in a cartridge project, points the CPU at it on reset, and puts a color on screen for the first time.

CHAPTER 5 · PART II

Your First Cartridge

> A raw `.BIN` is code and data. A cartridge is a game. This chapter builds the smallest possible one: a project that boots straight to a solid color on the emulated 320×200 display.

The project manifest

A cartridge project is simply a directory containing `FANTICON.CFG`: plain ASCII, one `KEY=VALUE` line at a time. Create a folder — say `/hello` — and inside it a manifest:

```
TITLE=HELLO FANTICON
ID=1A2B3C4D5E6F7788
MAIN=MAIN.ASM
OUTPUT=HELLO.FCN
SAVE_BANKS=0
MACHINE=1.0
```

Key	Rule
`TITLE`	1–22 printable ASCII characters; spaces allowed
`ID`	Exactly 16 hex digits, never zero — a stable save-file identity
`MAIN`	8.3 path to the root assembly source
`OUTPUT`	8.3 `.FCN` output filename
`SAVE_BANKS`	0–4 battery-backed 16 KiB banks
`MACHINE`	Required hardware version — `1.0` for v0.1

Inside Fanticon, `NEW PROJECT` scaffolds this manifest for you and generates a cryptographically random, nonzero 64-bit `ID` once; ordinary rebuilds never change it, because that ID is what ties a save file to this game for life. Unknown keys are a build error, so keep the manifest to exactly these six.

Sections: BANK and FIXED

Cartridge assembly adds two section directives on top of everything from Chapter 4. `FIXED` selects the 16 KiB image that is always mapped at `$C000-$FFFF` — your reset code and vectors live here. `BANK 0-255` selects a switchable 16 KiB image at `$8000-$BFFF` for everything too big to fit in fixed ROM. A minimal cartridge only needs `FIXED`.

MAIN.ASM

Create `MAIN.ASM` next to the manifest:

```
VCTRL    EQU   $C011
BGCOLOR  EQU   $C012

         FIXED
         ORG   $C100
RESET    SEI
         LDX   #$FF
         TXS
         LDA   #$00
         STA   VCTRL       ; background + sprites off
         LDA   #$3B
         STA   BGCOLOR     ; backdrop shows through -- pick any RGB332 byte
         CLI
IDLE     JMP   IDLE

         ORG   $FFFA
         DA    RESET,RESET,RESET   ; NMI, RESET, IRQ/BRK
```

Every cartridge's fixed ROM must supply all three CPU vectors, even ones it never uses. With `VIDEO_CONTROL` (`$C011`) set to zero, both the background and sprite layers are disabled and the display shows nothing but `BACKDROP_COLOR` (`$C012`) — a full 8-bit palette index, so all 256 programmable colors are available even with everything else turned off. `RESET` must live in fixed ROM or main RAM, and reset code cannot assume any switchable bank other than bank 0 is mapped yet — which is exactly why this program never touches `BANK_NUMBER` at all.

Build and run it

From `/hello`, type:

```
RUN
```

This builds the manifest into a validated `HELLO.FCN` and launches it immediately — the same action as **Build & Run** (`Shift+F5`) in the editor. The window switches to the 320×200 game display and fills with your chosen color. Press `Escape` to return to the editor.

Use `BUILD` alone to package without launching, and `RUN GAME.FCN` to validate and launch an existing image directly — useful once you start keeping built cartridges around separately from their source projects.

WHAT JUST GOT VALIDATED

Building a project does more than assemble source. The packager writes the fixed image followed by every referenced bank, fills every unwritten ROM byte with `$FF`, and writes a 64-byte header containing bank counts, the machine version, your title, your cartridge ID, and CRC-32 checksums of the fixed and banked ROM. A loader validates all of it — magic bytes, version, flags, bank counts, a nonzero ID, exact file size, every checksum, and the presence of all three vectors — before a single byte reaches the CPU. Chapter 20 covers the full format.

You have a running cartridge. The rest of this book is almost entirely about what you can put inside `RESET` and the interrupt handlers next to it — graphics, sound, input, and timing. First, meet the tool that lets you watch this program think, one instruction at a time.

CHAPTER 6 · PART II

The Debugger

> Fanticon's debugger is editor-owned: it starts with Build & Run, stops the entire virtual machine cold at a breakpoint, and never advances CPU, video, timer, input, or audio state while you are looking at it — so nothing keeps running behind your back.

Starting a session

Open `MAIN.ASM` from Chapter 5 and press `F9` on the `STA BGCOLOR` line to set a breakpoint — the source gutter marks it with `@`. Press `F5` (Build & Run). The VM runs at full speed and immediately stops on your breakpoint, marked with an arrow in the gutter.

Key	Debug action
`F5`	Build & Run when stopped; continue when paused
`F6`	Pause a running editor-owned game
`Shift+F5`	Stop the debug session
`F9`	Toggle a source breakpoint on the current line
`F10`	Step over `JSR`, or step one instruction otherwise
`F11`	Step one 6502 instruction
`Shift+F11`	Run until the current subroutine returns
`Ctrl/Cmd+F11`	Step one CPU/bus cycle

A breakpoint placed on a label or other non-emitting line resolves forward to the next line that emits bytes, so you can drop breakpoints on labels without thinking about it.

What the paused panel tells you

While stopped, Fanticon shows CPU registers and flags, stack bytes, memory around the program counter, the currently mapped bank, IRQ state, raster position, every APU channel's control/timer state and current output, the stop reason, and the eight most recently executed instructions. Step with `F11` and watch `BGCOLOR`'s value land in `A`, then get written on the following `STA`.

Watchpoints and raster breakpoints

The **Debug** menu also creates 16-bit memory-read and memory-write watchpoints, plus raster breakpoints entered as `LINE,DOT` — useful once you are chasing an effect timed to an exact scanline in Chapter 12. Debugger raster stops land on CPU cycle boundaries: every two video dots in machine 1.0.

> **WHAT THE DEBUGGER GIVES YOU**
>
> CPU instruction breakpoints, memory read/write watchpoints, single-instruction and single-cycle stepping, full register/stack/bank/IRQ/raster/APU inspection, raster-position breakpoints, and a complete VM pause — enough to build a debugging workflow around any cartridge you are developing.

You can now write, assemble, run, and step through 6502 code with confidence. Part III puts something worth looking at on that screen — starting with how Fanticon's video hardware works.

CART 03

03

GRAPHICS PROGRAMMING

The video hardware, the visual graphics editor, tile worlds, sprites, bitmap mode, and racing the raster beam.

CH 7 VIDEO TOUR · CH 8 GRAPHICS EDITOR · CH 9 TILES & SCROLLING · CH 10 SPRITES · CH 11 BITMAP MODE · CH 12 RASTER TRICKS

CHAPTER 7 · PART III

Video Hardware Tour

> Fanticon always outputs a fixed 320×200 indexed-color image at 60 Hz. Every game picks one background mode and may overlay the same 32 hardware sprites over either of them.

TABLE 7.1 — VIDEO_MODE ($C010)

Value	Mode
0	Blank — backdrop color only
1	320×200 viewport into a 64×32 map of 8×8, 4-bpp tiles
2	Packed 320×200, 4-bpp bitmap

This is one background layer. Tile mode is the efficient default for action games; bitmap mode suits illustration-heavy adventures, paint-style games, and software-rendered effects. Sprites work identically over either one.

The palette

There are 256 palette entries, arranged as 16 banks of 16 colors, each entry one RGB332 byte. Tile and sprite pixels store a 4-bit color number and select one of the 16 banks; a whole bitmap shares a single bank chosen by `BITMAP_PALETTE`. Color 0 is **transparent** for sprites but **opaque** for backgrounds.

```
PALINDEX EQU   $C01B
PALDATA  EQU   $C01C

         LDA   #$20       ; start at bank 2, entry 0
         STA   PALINDEX
         LDA   #%11100000 ; RGB332: R=7 G=0 B=0 -- bright red
         STA   PALDATA    ; index auto-increments after every read or write
```

At reset, entry `N` already contains RGB332 byte `N` — an identity palette. Components expand to full 8-bit color with rounding: `R=round(R3×255/7)`, `G=round(G3×255/7)`, `B=round(B2×255/3)`, before the host's CRT presentation pass runs.

Video registers at a glance

Address	Name	Purpose
`$C010`	`VIDEO_MODE`	Blank, tile, or bitmap background
`$C011`	`VIDEO_CONTROL`	Bit 0 background enable, bit 1 sprite enable
`$C012`	`BACKDROP_COLOR`	Full 8-bit palette index shown when the background is off
`$C013–$C016`	`SCROLL_X` / `SCROLL_Y`	Signed 16-bit tilemap scroll
`$C017–$C01A`	`RASTER_X` / `RASTER_Y`	Raster-compare target
`$C01B/$C01C`	`PALETTE_INDEX` / `DATA`	Palette read/write with auto-increment
`$C01D`	`BITMAP_PALETTE`	Palette bank for the whole bitmap
`$C01E`	`VIDEO_STATUS`	Live VBlank/HBlank, latched sprite overflow

Bit	`VIDEO_STATUS` field
0	Live VBlank
1	Live HBlank
2	Sprite overflow, latched for the current frame
7–3	Reserved

HBlank covers dots 320–399; VBlank covers lines 200–261; overflow clears at line 0, dot 0. Reading the register clears nothing — you explicitly acknowledge interrupt sources instead, as Chapter 17 covers.

MASTER TIMING

60 Hz frames, 262 scanlines each, 400 dots per scanline, 320×200 visible. The video dot clock is 6.288 MHz — exactly two dots per 3.144 MHz CPU cycle — giving 200 CPU cycles per scanline and 52,400 per frame. VRAM is dual-ported: v0.1 has no CPU wait states or display-induced bus contention, so writes land exactly where you time them, and never stall your program.

Chapter 8 introduces the visual tool you will use to author tiles, maps, sprite art, and palettes, before Chapters 9–11 put each mode to work.

CHAPTER 8 · PART III

The Graphics Editor

> Fanticon graphics are edited as `.GFX` documents, right in the normal editor tab strip, alongside a shared `.PAL` palette document that every image can reference. Both are plain ASCII, valid assembler input, and editable visually *or* as source with `Ctrl/Cmd+G`.

Creating one

Choose **File > New Graphics** (`Ctrl/Cmd+Shift+N`). A new `.GFX` file references `GAME.PAL`; if that file does not exist yet, the editor creates a default DawnBringer DB16 palette automatically. DB16 color 0 maps to RGB332 `$00` (true black) rather than quantizing DB16's very dark purple, which would otherwise pick up an unwanted red cast.

One shared library, three views

There is exactly one library of 256 reusable 8×8 patterns shared by everything on screen:

1. **Pattern** edits one reusable 8×8 image.
2. **Map** places those patterns into the 64×32 circular background.
3. **16×16 Sprite** edits four consecutive patterns together as one composite.

Change a pattern and every map cell and sprite referencing it updates — there is only one copy. Press number keys to switch views: `1` pattern sheet, `2` the pannable 40×25 map viewport, `3` the 16×16 sprite composite, `4` the palette, `5` the packed 320×200 bitmap. Drawing tools are `P` Pencil, `F` connected Fill, and `I` Eyedropper, with `H`/`V` flips and `R` rotation in Pattern view.

THE CIRCULAR MAP

In Map view, arrow keys pan the viewport one tile at a time and **wrap at all four map edges** — every one of the 64×32 cells is directly reachable and editable, including the seam. That circularity is not just an editor convenience; it is exactly how the hardware scroll registers work, which is what makes infinite scrolling possible in Chapter 9.

Palette banks in practice

A typical project assigns palette banks by role, not by image:

```
bank 0  UI and text
bank 1  player
bank 2  enemies
bank 3  current level
bank 8  title bitmap
```

Palette view offers `DB16`, `PICO-8`, `C64`, and `EGA` presets — click one or press `N`/`Shift+N` to replace just the selected bank. `R`/`G`/`B` nudge a color's RGB332 component up, Shift down. Every edit is undoable.

What gets saved

A new tilemap-mode `.GFX` file begins:

```
;@FANTICON-GFX 3
;@PALETTE-FILE GAME.PAL
;@MODE TILEMAP
```

Saving `WORLD.GFX` generates three hardware-native blocks under `;@TILES`, `;@MAP`, and `;@ATTRIBUTES` markers, exposed to your code as the labels `WORLD_CHR`, `WORLD_MAP`, and `WORLD_ATR`. A palette file exposes one label, e.g. `GAME_PAL`, after its own `;@FANTICON-PAL 1` header. Because both are ordinary assembler source, pulling them into a cartridge is one line each:

```
        FIXED
        ORG   $D000
        PUT   GAME.PAL
        PUT   WORLD.GFX
```

Visual saves always produce canonical uppercase hex, and returning from the ASCII source view validates every marker and block size — malformed data stays visible with a readable error instead of being silently truncated. Because palette references are explicit text, sharing and changing palettes stays visible in ordinary source control diffs.

Loading the palette at runtime

`GAME_PAL` is the complete 256-byte table — all 16 banks, back to back — so loading it is one pass through `PALINDEX`/`PALDATA`, letting the auto-increment do the addressing for you. Unlike tile and sprite data, the palette registers sit in the fixed `$C000–$C0FF` I/O page, not the bank window, so no bank switch is needed before or during this loop:

```
PALINDEX EQU   $C01B
PALDATA  EQU   $C01C

         LDA   #$00       ; start at bank 0, entry 0
         STA   PALINDEX
         LDX   #$00
.LOOP    LDA   GAME_PAL,X
         STA   PALDATA     ; writes entry X, then PALINDEX advances on its own
         INX
         BNE   .LOOP       ; X wraps 255->0, ending the loop after exactly 256
writes
```

The `INX`/`BNE` pair is the standard 6502 idiom for a full 256-byte pass: `X` counts `0` through `255` and wrapping back to `0` clears the zero flag's complement, so `BNE` falls through exactly once, after the 256th byte. The same loop works for a partial table — compare `CPX` against the byte count instead of relying on the wrap — which is what you want when loading just one or two banks rather than the whole file.

TRY IT

Open `/demos/graphics/SCENE.GFX` and `GAME.PAL`. Pan the map with arrow keys, then open `MAIN.ASM` in the same folder to see the complete runtime path from ROM to screen: `PUT` the palette and graphics resources, upload the palette through `PALDATA`, stage GFX pages through RAM, copy patterns/map/attributes to VRAM, and finally build a sprite from patterns 4–7. Run it with `RUN` or `F5`.

CHAPTER 9 · PART III

Tiles & Scrolling Worlds

> Tile mode VRAM holds a pattern library, a 64×32 tile-number map, and a matching attribute map — 12,288 bytes total, small enough to fit in a single 16 KiB cartridge bank.

VRAM offset	Size	Contents
`$0000–$1FFF`	8 KiB	256 packed 4-bpp, 8×8 tile patterns
`$2000–$27FF`	2,048 B	64×32 tile-number map
`$2800–$2FFF`	2,048 B	64×32 tile-attribute map
`$3000–$30FF`	256 B	32 eight-byte sprite records

Every pattern byte packs two pixels — high nibble left, low nibble right — so no runtime unpacking is ever needed:

```
tile pattern offset = tile_number × 32
tilemap offset      = tile_y × 64 + tile_x
tile number byte    = $2000 + tilemap offset
tile attribute byte = $2800 + tilemap offset
```

Each attribute byte packs five fields:

Bit	Field
7	Reserved
6	Foreground priority
5	Vertical flip
4	Horizontal flip
3–0	Palette bank, 0–15

A tile's foreground-priority bit lets it cover every sprite regardless of that sprite's own behind-background flag — the one hard rule for layering platforms above characters.

Loading a set at runtime

Copy `WORLD_CHR` to VRAM `$0000`, `WORLD_MAP` to `$2000`, and `WORLD_ATR` to `$2800` — exact sizes `$2000`, `$0800`, and `$0800`. All three live in VRAM bank 0, alongside the sprite table from Chapter 10. Because banked ROM and VRAM share the same `$8000–$BFFF` window, a loader stages each piece through main RAM: map the ROM bank in, copy it out to RAM, map VRAM bank 0 in, copy from RAM into it. Chapter 19 builds a small reusable `PAGECOPY` routine for exactly this relay; here it is loading `WORLD_MAP` from a switchable cartridge bank, the way a full multi-level game would keep its tile sets:

```
BANKKIND EQU   $C000
BANKNUM  EQU   $C001
VMODE    EQU   $C010
VCTRL    EQU   $C011

         ; leg 1: WORLD_MAP's cartridge bank -> the STAGE buffer in RAM
         LDA   #$00
         STA   BANKKIND     ; cartridge ROM
         LDA   #BANKOF(WORLD_MAP)
         STA   BANKNUM
         LDA   #<WORLD_MAP
         STA   SRC
         LDA   #>WORLD_MAP
         STA   SRC+1
         LDA   #<STAGE
         STA   DST
         LDA   #>STAGE
         STA   DST+1
         LDA   #8           ; the tile-number map is 2,048 bytes = 8 pages
         JSR   PAGECOPY

         ; leg 2: STAGE -> the tile-number map in VRAM bank 0
         LDA   #$02
         STA   BANKKIND     ; video RAM
         LDA   #$00
         STA   BANKNUM      ; patterns, map, attributes, and sprites all live in
bank 0
         LDA   #<STAGE
         STA   SRC
         LDA   #>STAGE
         STA   SRC+1
         LDA   #<$A000    ; $8000 + $2000, the tile-number map's offset
         STA   DST
         LDA   #>$A000
         STA   DST+1
         LDA   #8
         JSR   PAGECOPY

         ; WORLD_CHR (8 KiB = 32 pages, -> $8000) and WORLD_ATR (8 pages,
         ; -> $A800) load the same way -- change SRC, DST, and the page count

         LDA   #1
         STA   VMODE        ; tilemap mode
         LDA   #%00000011
         STA   VCTRL        ; background + sprites on
```

Scrolling that never ends

`SCROLL_X` and `SCROLL_Y` are independent signed 16-bit values that wrap the viewport across a 512×256-pixel circular tile map. Increasing X moves the viewport right; increasing Y moves it down. Decrementing zero to `$FFFF` scrolls smoothly left or up by one pixel — the wrap is seamless in both directions.

Treat the 64×32 map as a ring buffer for a world larger than it: keep world-space camera coordinates in RAM, write their low 16 bits to the scroll registers, and map a world tile `(wx,wy)` to hardware cell `(wx & 63, wy & 31)`. A fully scrolled 320×200 viewport occupies at most 41×26 cells, leaving at least 23 hidden columns and 6 hidden rows — plenty of room to repaint the next row or column of the map during VBlank, just before it scrolls into view.

TRY IT

`/demos/tiles` builds a full 64×32 tilemap and uses a VBlank IRQ to sample controller 1: arrow keys scroll freely in every direction, and the 16-bit scroll registers wrap smoothly around the whole map with no visible seam. Read `MAIN.ASM` to see the VBlank-driven input pattern you will reuse constantly starting in Chapter 17.

CHAPTER 10 · PART III

Sprites

> Fanticon gives you 32 hardware sprites, each an eight-byte record at VRAM `$3000 + sprite_number × 8`, evaluated in record order every scanline.

Offset	Name	Meaning
0	`X_LOW`	X position bits 0–7
1	`X_FLAGS`	Bit 0: X bit 8. Bit 1: behind-background priority
2	`Y`	Y position
3	`TILE`	First pattern tile
4	`ATTR`	Enable, size, flips, and palette — see below

Bit	`ATTR` field
7	Enable
6	16×16 size (0 = 8×8)
5	Vertical flip
4	Horizontal flip
3–0	Palette bank, 0–15

An 8×8 sprite uses one pattern tile. A 16×16 sprite uses four consecutive tiles arranged `N,N+1` above `N+2,N+3`, where `N` must be divisible by four; flips apply to the whole composite at once.

Clipping and negative coordinates

X values `$1F0–$1FF` mean −16 through −1; Y values `$F0–$FF` mean the same. This lets an object enter smoothly from the left or top edge without special-casing anything — sprites clip at all four screen edges and never wrap. Here is a complete minimal sprite setup, in the same spirit as `/demos/sprites`:

```
XPOS    EQU   $20
XFLAG   EQU   $21
YPOS    EQU   $22
REC0    EQU   $B000   ; sprite 0's record, bank window + $3000

        LDA   #156
        STA   XPOS
        STA   REC0        ; X_LOW
        LDA   #0
        STA   XFLAG
        STA   REC0+1      ; X bit 8 clear, in front of background
        LDA   #96
        STA   YPOS
        STA   REC0+2
        LDA   #1
        STA   REC0+3      ; first tile
        LDA   #%10000010
        STA   REC0+4      ; enabled, 8x8, palette bank 2
```

Moving the sprite once per VBlank and letting `X` cross `$000` from below naturally produces `$1FF` (−1); the renderer clips it at the edge rather than wrapping it around, the same way it would leave the visible edge of the display.

Priority and the eight-per-line limit

Sprite color 0 is transparent. A background tile's foreground-priority attribute covers every sprite outright; otherwise a sprite's own behind-background flag puts it under a nonzero background pixel, and a front sprite simply wins ties. Background color 0 never hides a sprite. Lower record numbers always win overlap ties against other sprites.

Only the **first eight enabled sprites** whose vertical range intersects a scanline are drawn on it — every enabled sprite counts toward that limit even if it is horizontally off-screen or fully transparent on that line. A ninth candidate sets the sprite-overflow status bit (`VIDEO_STATUS` bit 2) and is simply skipped for that line. Gameplay collision is handled in your own code, comparing object bounds against each other rather than rendered pixels.

Sprite records are sampled at the *start* of each scanline, so writing a record mid-line affects only the following scanline — a simple, deterministic rule once you start multiplexing sprites for effects beyond 32 on-screen objects.

TRY IT

`/demos/sprites` shows exactly this pattern: one 8×8 sprite, moved once per VBlank by arrow keys. Push it past any edge to watch clipping and the signed coordinate wraparound in action.

Sprite limits and flicker

Fanticon's sprite hardware has two separate limits, and they run out in different ways:

Limit	What happens past it
32 records total	There is no 33rd record — an object with no assigned record simply has no sprite to draw with
8 drawn per scanline	The ninth-and-later enabled, vertically-intersecting record on a line is skipped for that line only, and `VIDEO_STATUS` bit 2 latches

Both limits count enabled records whose vertical range intersects a line, whether or not they are horizontally visible or transparent there — a wide spread of off-screen bullets can still burn through the eight-per-line budget of an on-screen line at the same height.

Rotating who wins the race

Because the hardware always favors the *lowest* record number on a contested scanline, a fixed object-to-record assignment means the same low-priority objects silently vanish every time a line gets crowded. Flicker fixes that by rotating which object occupies which record every frame, so the dropout moves around instead of sitting permanently on one object — the same technique NES games used to show more than eight objects on a busy line. Each frame, one call before writing sprite records:

```
FLICKER  EQU   $30      ; rotation offset, one byte
OBJCOUNT EQU   $31      ; active objects this frame, 1-32
OBJIDX   EQU   $32      ; which object goes into the record being written

; advance the rotation by one object per frame, wrapping at OBJCOUNT
         INC   FLICKER
         LDA   FLICKER
         CMP   OBJCOUNT
         BCC   .SKIP
         LDA   #0
         STA   FLICKER
.SKIP
         LDA   FLICKER
         STA   OBJIDX
         LDX   #0            ; X = sprite record index, 0-31
.LOOP    LDY   OBJIDX
         JSR   WRITERECORD   ; your routine: object Y -> sprite record X
         INC   OBJIDX
         LDA   OBJIDX
         CMP   OBJCOUNT
         BCC   .NOWRAP
         LDA   #0
         STA   OBJIDX
.NOWRAP  INX
         CPX   OBJCOUNT
         BCC   .LOOP
```

Record 0 — the one every contested scanline favors — holds a different object each frame, so a crowded line drops a different object each time rather than always the last one in the list. If `OBJCOUNT` is below 32, disable the leftover records explicitly (clear `ATTR` bit 7) once when the count drops, or a stale record from a busier frame keeps drawing.

CAP THE PROBLEM BEFORE YOU ROTATE IT

Flicker distributes dropout; it does not remove it. A line that regularly needs 20 sprites will still flicker heavily no matter how the rotation is arranged. Keep genuinely high-density effects — explosions, bullet spreads — short-lived and staggered in Y, and consider drawing the least important layer (dust, sparks) as tiles instead of sprites when a scene is already sprite-heavy.

CHAPTER 11 · PART III

Bitmap Mode

> Bitmap mode trades the tile grid for 32,000 directly addressable 4-bpp pixels — the mode for illustration, paint tools, and effects that do not fit neatly into 8×8 cells.

VRAM offset	Bank	Contents
`$4000-$7FFF`	1	First bitmap section (16 KiB)
`$8000-$BCFF`	2	Remaining bitmap pixels (15,616 B)

```
VRAM byte offset = $4000 + Y × 160 + (X / 2)
even X pixel = high nibble
odd  X pixel = low nibble
```

The 4-bit pixel combines with `BITMAP_PALETTE` (`$C01D`, low nibble) to select one of the 256 palette entries, the same bank for every pixel in the bitmap. Sprite patterns and records stay in VRAM bank 0, untouched by bitmap data, so sprites composite over bitmap graphics exactly as they do over tiles. Scroll registers have no effect in this mode.

Loading a bitmap set

The graphics editor packages a bitmap `.GFX` as three consecutive cartridge banks. Starting at bank 7, for example:

Cartridge bank	Label	Destination
7	`TITLE_BM0`	First bitmap portion → VRAM bank 1
8	`TITLE_BM1`	Remaining pixels → VRAM bank 2
9	`TITLE_CHR`	Resident sprite patterns → VRAM bank 0

Include the bitmap `.GFX` before your program's `FIXED` section, then explicitly restore `FIXED` afterward for ordinary code. Load its referenced `.PAL`, set `BITMAP_PALETTE`, select `VIDEO_MODE=2`, and enable sprites as needed — the editor limits the starting bank to 0–253 so all three banks always fit.

WRITING 32,000 BYTES TAKES TIME

Filling a full-screen bitmap from the 6502 is substantial work: at roughly 4–6 cycles per byte for a simple copy loop, painting all 32,000 bytes costs on the order of one to two full frames. `/demos/bitmap` demonstrates exactly this — startup visibly takes a few frames while the CPU writes the whole packed bitmap through both VRAM banks. Budget for it, and consider painting during a loading or title screen rather than a moment that needs to feel instant.

Bitmap mode and tile mode are not exclusive within a project — a title screen in bitmap mode and the rest of the game in tile mode is a completely normal combination, since switching `VIDEO_MODE` is a single register write.

CHAPTER 12 · PART III

Raster Tricks

> Because Fanticon is cycle-deterministic, your code can race the raster beam on purpose — changing colors or scroll position at an exact scanline, well before general-purpose GPUs made that unnecessary.

The raster comparator

`RASTER_X` (`$C017/$C018`, 0–399) and `RASTER_Y` (`$C019/$C01A`, 0–261) name one exact dot. When the beam visits it, `IRQ_PENDING` bit 1 sets — once per visit. Clearing it while the beam is still sitting on that coordinate does not retrigger it; programming a new target ahead of the beam can still fire this frame, while a target the beam already passed waits for the next one. Reset selects the unreachable target `(511,511)`, so a cartridge that never touches these registers never gets a spurious raster interrupt.

```
RASTX_LO EQU $C017
RASTY_LO EQU $C019
IRQEN    EQU $C003

         LDA   #0
         STA   RASTX_LO     ; dot 0
         LDA   #50
         STA   RASTY_LO     ; line 50 -- high bytes stay 0 here
         LDA   #%00000010
         STA   IRQEN        ; enable the raster-compare source
```

Color bands with zero framebuffer cost

The simplest raster trick needs no VRAM writes at all: reprogram `BACKDROP_COLOR` from a raster IRQ handler to paint horizontal bands. Each time the handler fires, it writes a new color and re-arms the comparator for the next band boundary — `/demos/raster` does exactly this to draw four solid bands using only the dot/line comparator and raster IRQs, without touching a single framebuffer byte.

Budgeting cycles against the beam

Finer effects — a palette split mid-scanline, or bending scroll position along a curve one line at a time — have to win a race against the beam itself. HBlank covers dots 320–399, which at two dots per CPU cycle is exactly 40 CPU cycles: enough for IRQ entry, a table lookup, and one register write, but not much more.

`/demos/wave` uses one raster IRQ per visible scanline to bend the tilemap's horizontal scroll along a smooth 128-step sine table, with vertical scroll following the same curve once per frame at a quarter-cycle offset — keeping both axes moving without ever folding at a scanline boundary. Its raster comparator deliberately fires *before* HBlank begins, so that IRQ entry and the table lookup consume exactly enough cycles that `SCROLL_X` gets written during HBlank itself, before the next visible line starts fetching. That is the general pattern for any per-scanline raster effect: trigger the interrupt slightly early, and let the handler's own fixed cost land the write inside the blanking window you are aiming for.

SAME-CYCLE ORDERING IS A GUARANTEE, NOT LUCK

Within every CPU cycle, the raster fetches the first dot, your bus transfer completes on the second dot, mapped writes become visible, and only then does the raster fetch that second dot — with new interrupt-pending bits set last of all. A same-cycle `IRQ_PENDING` clear can never erase an event that just happened. This exact, reproducible ordering is what makes racing the beam a reliable technique on Fanticon rather than a fragile trick that only works on one host.

You can now put pixels, characters, sprites, and beam-timed effects on screen. Part IV turns to the other half of a console’s personality — its four voices of sound.

CART 04

04

SOUND

Four deterministic voices, direct register programming, and the visual tracker that composes for them.

CH 13 AUDIO TOUR · CH 14 DIRECT REGISTERS · CH 15 MUSIC TRACKER

CHAPTER 13 · PART IV

Audio Hardware Tour

> Fanticon has four deterministic, monophonic voices: two pulse channels, one triangle channel, and one noise channel — the sound character of the NES. Envelopes, sweeps, and note timing are all under direct control of your 6502 code, driven register by register.

Range	Voice
`$C030–$C033`	Pulse 1
`$C034–$C037`	Pulse 2
`$C038–$C03B`	Triangle
`$C03C–$C03F`	Noise
`$C040`	Master control

Pulse channels

Pulse 1 and Pulse 2 are identical, each four registers: control, an 11-bit timer split across two bytes, and a phase-reset register that any write resets. The control byte packs three fields:

Bit	Control field
7	Enable
6–5	Duty, 0–3 — see table below
4	Reserved
3–0	Volume, 0–15

Four duty sequences are available:

Duty	Width	Sequence
0	12.5%	`0 1 0 0 0 0 0 0`
1	25%	`0 1 1 0 0 0 0 0`
2	50%	`0 1 1 1 1 0 0 0`
3	75%	`1 0 0 1 1 1 1 1`

```
pulse Hz = 3,144,000 / (16 × (timer + 1))
```

Triangle

One 32-step, 4-bit sequence at a fixed amplitude, scaled only by master volume — matching its main historical inspiration exactly:

```
triangle Hz = 3,144,000 / (32 × (timer + 1))
```

Noise

A 15-bit right-shifting LFSR, controlled by one `NOISE_CONTROL` (`$C03C`) byte:

Bit	Control field
7	Enable
6	Short mode (0 = long mode)
5–4	Reserved
3–0	Volume, 0–15

Long mode feeds `bit0 XOR bit1` back into bit 14 and repeats after 32,767 shifts — ordinary percussion. Short mode feeds `bit0 XOR bit6` instead and repeats after only 93 shifts from reset, giving a recognizably metallic, pitched noise. A set bit 0 of the LFSR outputs zero; a clear bit 0 outputs the channel's 4-bit volume. Sixteen fixed periods (index in `NOISE_PERIOD` bits 0–3) span roughly 440 Hz to 449 kHz shift rates, scaled from the NTSC NES table to Fanticon's clock.

Enable, phase, and the divider

A channel's enable bit only gates whether it reaches the mixer — the oscillator and divider keep running underneath even while muted, so re-enabling resumes wherever the hardware silently got to. Writing a timer half changes the reload value but does not reload the running divider; only a phase-reset write snaps the waveform back to step zero and reloads the divider from the current timer. Use phase reset whenever a note needs a synchronized attack.

Mixing

One `AUDIO_MASTER` (`$C040`) byte gates and scales all four voices together:

Bit	Control field
7	Master enable
6–4	Reserved
3–0	Master volume, 0–15

The two pulse DACs share one nonlinear path; triangle and noise share a second. For current 4-bit levels `p1`, `p2`, `t`, `n`:

```
pulse = 95.88 / (8128 / (p1 + p2) + 100)
tnd   = 159.79 / (1 / (t/8227 + n/12241) + 100)
mix   = (pulse + tnd) × master_volume / 15
```

Fanticon evaluates the integer-rational equivalent of these equations, so results are bit-identical everywhere the machine runs — the same NES-like nonlinear balance on every system. The host then applies a light, presentation-only stereo width and a short, subdued reverb; neither can feed back into APU state, consume CPU cycles, or change VM timing.

RESET SILENCES EVERYTHING

Power-on, CPU RESET, and cartridge removal all perform the same APU reset: every register zero, every channel and master enable clear, pulse/triangle phase at step zero, every divider zero, and the noise LFSR reseeded to 1. An ordinary pause instead freezes emulated time and all APU state exactly as it stood — it does not reset anything.

Chapter 14 turns these registers into sound.

CHAPTER 14 · PART IV

Programming Sound Directly

> Every voice follows the same four-step pattern: pick a timer for the pitch you want, set volume and duty/mode, reset phase for a clean attack, and enable the channel — plus one master switch that gates everything.

```
P1_CTRL  EQU   $C030
P1_TLO   EQU   $C031
P1_THI   EQU   $C032
P1_RST   EQU   $C033
MASTER   EQU   $C040

; Concert A (~440 Hz): timer = 3,144,000/(16*440) - 1 = 446 = $1BE
         LDA   #$BE        ; timer low
         STA   P1_TLO
         LDA   #$01        ; timer high (11-bit total)
         STA   P1_THI
         LDA   #%10101111 ; enable, duty 2 (50%), volume 15
         STA   P1_CTRL
         STA   P1_RST       ; any write resets phase for a clean attack
         LDA   #%10001111 ; master enable, volume 15
         STA   MASTER
```

ULTRASONIC TIMERS ARE ALLOWED, NOT SILENCED

Timer values `$000–$007` are legal and produce ultrasonic fundamentals rather than being clamped or muted — they simply are not useful pitches for music. Double-check a computed timer value before trusting a very high note.

Driving sound from a timer, not the main loop

Audio registers hold whatever they were last written and can be read back, but nothing advances them automatically — a game normally updates volume and timer values from a VBlank interrupt or a Fanticon interval timer (Chapter 18), the same way it drives animation. `/demos/audio` plays both pulses, the triangle, and noise together, using a VBlank IRQ to change the lead note and the backdrop color in lockstep, four times per cycle — a compact demonstration of interrupt-driven sound without a full tracker.

Reference basis

The voice layout follows the Ricoh 2A03 tradition found in the NESdev community's APU documentation. Fanticon freezes its own exact waveform, divider, LFSR, and mixer choices as its own behavior — not a claim of being electrically identical to any specific NES revision.

Hand-writing register pokes works well for sound effects and simple leads. For a complete composition, Chapter 15's tracker generates this same register stream for you.

CHAPTER 15 · PART IV

The Music Tracker

> Choose **File > New Music** or open an existing `.MUS` file to enter Fanticon's four-channel tracker — one column per voice, compiled straight into assembler source your cartridge includes directly.

Three views, one song

Press `V` to rotate through them, or click their tabs:

- **Pattern** — notes, instrument, and volume per row. The active row stays highlighted and centered during playback.
- **Frames** — the song order. Each frame independently selects a pattern for pulse 1, pulse 2, triangle, and noise, FamiTracker-style, so one channel can repeat a pattern while another advances.
- **Instrument** — volume, arpeggio, pitch, and tone/duty envelopes.

Notes enter on a piano-style layout, `A W S E D F T G Y H U J` across a chromatic octave, with `Z`/`X` shifting the entry octave. `OFF` disables a channel for that row; `---` holds the previous value. Noise notes `N00`–`N0F` pick its sixteen periods directly. `[`/`]` move between frames without leaving Pattern view.

Instruments are four independent sequences

Each sequence advances once per *video frame*, including the frames between tracker rows — this is where a Fanticon instrument gets its character:

Sequence	Values
Volume	`0–15`, scales the row's volume
Arpeggio	Signed semitone offsets from the note
Pitch	Signed timer offsets, for slides and vibrato
Tone/Duty	Pulse duty 0–3; on noise, 0 selects long mode and nonzero selects short/metallic; triangle ignores it

Playing it back in a cartridge

`/demos/music` includes `PLAYER.INC`, a small reference player. Pass the compiled song header in `X`/`Y` once, then tick it every VBlank:

```
        LDX   #<SONG_MUSIC
        LDY   #>SONG_MUSIC
        JSR   MUSIC_START
; ...inside your VBlank handler, once per frame...
        JSR   MUSIC_TICK
```

Each tick decodes one delta frame straight into `$C030-$C03E`. That decode is deliberately small and predictable — all the expensive pattern and envelope work already happened at build time, not at runtime. `MUSIC_STOP` disables every voice. The driver reserves zero-page `$30-$41`; relocate its `EQU` definitions if that range collides with your own variables.

INCLUDED DEMO SONG

`/demos/music/SONG.MUS` is a complete chiptune arrangement of Beethoven's "Ode to Joy" (Mutopia Music ID 528, public domain), with a Fanticon-original noise percussion part. Space starts and stops playback in the tracker. Opening it also demonstrates automatic migration: it is stored in the legacy flat-row v1 format, and saving it upgrades the file to v2 patterns, frames, and instruments in place.

Importing existing NES music

Editor mode can run an isolated NSF driver and transcribe its output directly into an editable `.MUS` resource:

```
NSF2MUS INPUT.NSF OUTPUT.MUS [TRACK]
```

The importer samples all four supported channels at 60 Hz, converts NES timer values to the nearest Fanticon note, and preserves volume, duty, noise mode, retriggers, and legato pitch changes, stopping automatically once its loop detector recognizes a repeating intro. An NSF is executable code rather than a score, so this is necessarily a transcription: fine pitch quantizes to Fanticon notes, PAL timing retunes to Fanticon's clock, and DPCM samples have no matching channel and are dropped with a warning rather than silently ignored.

With graphics and sound both in hand, Part V covers the remaining systems every game needs: controllers, interrupts, and precise timing.

CART 05

05

INPUT, INTERRUPTS & TIMING

Reading controllers, structuring interrupt handlers, and driving precise sub-frame schedules with the two hardware timers.

CH 16 CONTROLLERS · CH 17 INTERRUPTS · CH 18 TIMERS

CHAPTER 16 · PART V

Controllers

> Two standard eight-button controllers are readable in parallel through simple memory-mapped registers, keeping input code small and predictable.

Address	Name	Description
`$C050`	`PAD0_STATE`	Currently held controller-1 buttons
`$C051`	`PAD0_PRESSED`	Newly pressed buttons; read clears the returned bits
`$C052`	`PAD1_STATE`	Currently held controller-2 buttons
`$C053`	`PAD1_PRESSED`	Newly pressed buttons; read clears the returned bits

```
7       6       5       4       3       2       1       0
START   SELECT  B       A       RIGHT   LEFT    DOWN    UP
```

A set bit means pressed. Inputs are sampled once per frame, at line 0, dot 0. `PAD_PRESSED` accumulates newly pressed edges until you read it, then clears every bit it returned — a same-cycle new edge is latched *after* the read completes, so it is never lost even if you happen to read at exactly the wrong instant. Opposite directions may be held simultaneously; Fanticon does not resolve that for you, so decide your own policy (last-pressed-wins is common). A disconnected controller simply reads all-released.

```
PAD0     EQU   $C050
PAD0P    EQU   $C051
BTN_A    EQU   %00010000
BTN_RIGHT EQU  %00001000

; held-state check -- movement, usually read every frame
         LDA   PAD0
         AND   #BTN_RIGHT
         BEQ   NOTRIGHT
         INC   PLAYERX
NOTRIGHT

; press-edge check -- jump, menu confirm, anything that should fire once
         LDA   PAD0P
         AND   #BTN_A
         BEQ   NOJUMP
         JSR   STARTJUMP
NOJUMP
```

Defaults and hot-plug

Controller 1 defaults to arrow keys, `Z` for A, `X` for B, `Space` for Select, and `Enter` for Start, and combines its keyboard and gamepad state into one reading. Controller 2 has no default keyboard mapping and relies on a second gamepad. The first two connected gamepads keep stable controller slots across hot-plug events, and losing focus, a hot-unplug, or a debugger stop all release host input latches cleanly rather than leaving a button artificially stuck down.

READ ONCE, USE TWICE

Reading `PAD_PRESSED` clears its bits, so if two different systems in your game both need to know about the same button press this frame, read it once into a RAM variable at the top of your input handling and let both systems consult the copy.

Both registers are typically read from inside a VBlank handler, which Chapter 17 covers in detail next.

CHAPTER 17 · PART V

Interrupts & the IRQ Controller

> Four interrupt sources share one IRQ controller: VBlank, the raster comparator, and the two interval timers. All of them OR onto the 6502's single, level-sensitive IRQ pin, so your handler decides what matters first by checking bits in whatever order suits your game.

Bit	Mask	Source
0	`$01`	Vertical blank begins
1	`$02`	Raster X/Y comparison matches
2	`$04`	Timer 0 reaches zero
3	`$08`	Timer 1 reaches zero

`IRQ_ENABLE` (`$C003`) chooses which sources can assert the CPU's IRQ line at all; `IRQ_PENDING` (`$C002`) is read/write-one-to-clear — writing a 1 to a pending bit acknowledges that source, writing 0 leaves it alone, and disabling a source does not erase its already-pending state. Nothing is acknowledged automatically on interrupt entry; your handler always acknowledges explicitly, so it can tell apart which of possibly several simultaneously pending sources it needs to service.

A complete VBlank handler

The vast majority of Fanticon games structure almost everything — input, animation, sound, VRAM updates — inside one VBlank handler, because VBlank is the one moment guaranteed not to race the visible raster:

```
IRQPEND  EQU   $C002
IRQEN    EQU   $C003

         ORG   $FFFA
         DA    RESET,RESET,IRQHANDLER

IRQHANDLER
         PHA
         TXA
         PHA
         TYA
         PHA

         LDA   IRQPEND
         AND   #%00000001    ; is this a VBlank event?
         BEQ   NOTVBLANK

         LDA   #%00000001
         STA   IRQPEND      ; acknowledge VBlank -- write-one-to-clear
         JSR   READPADS
         JSR   UPDATEGAME
         JSR   MUSIC_TICK

NOTVBLANK
         PLA
         TAY
         PLA
         TAX
         PLA
         RTI
```

Preserving `A`, `X`, and `Y` on entry and restoring them before `RTI` is not optional discipline — interrupts can land in the middle of any instruction sequence in your main loop, and `RTI` restores the interrupted status flags on its own, so the temporary flag changes from your own `PLA`/transfer sequence never escape the handler.

NMI IS RESERVED

NMI (`$FFFA`) is falling-edge-sensitive and, on the 6502, cannot be masked the way IRQ can. No standard v0.1 device drives it — it is explicitly reserved for a future cartridge or debugging facility. Point the NMI vector at a safe handler (often the same one as RESET, or a simple `RTI`) and otherwise ignore it for now.

Combine sources by testing multiple bits, in priority order you choose:

```
          LDA   IRQPEND
          AND   #%00000100   ; Timer 0?
          BEQ   SKIPTIMER0
          LDA   #%00000100
          STA   IRQPEND      ; ack only this source
          JSR   ONTIMER0
SKIPTIMER0
```

Because IRQ is level-sensitive rather than edge-sensitive, an unacknowledged source keeps re-asserting the CPU's IRQ line — if you forget to acknowledge one, your handler simply runs again immediately after `RTI`. That is a useful debugging signal in its own right: a game that appears to hang inside its interrupt handler is almost always missing an acknowledgment somewhere.

CHAPTER 18 · PART V

Timers

> Two independent 16-bit down-counters tick once every CPU cycle, entirely apart from the raster — useful for music ticks, sub-frame scheduling, or timing input that needs finer resolution than once per frame.

Timer	Reload low	Reload high	Control	Count low	Count high
Timer 0	`$C060`	`$C061`	`$C062`	`$C063`	`$C064`
Timer 1	`$C068`	`$C069`	`$C06A`	`$C06B`	`$C06C`

Control mask	Meaning
`$01`	Enable counter
`$02`	Automatically reload after reaching zero

Reading count-low latches count-high until you read it, giving a coherent 16-bit snapshot the same way the frame counter does. Reload writes never disturb a running count — only an enable transition from 0 to 1 loads the reload value and starts counting, beginning on the following CPU cycle. Automatic mode reloads immediately at zero and keeps firing at an exact interval; one-shot mode simply stops at zero. Disabling a timer preserves its current count, so re-enabling later restarts cleanly from the reload value, not from wherever it happened to stop. A reload of zero means the maximum interval: 65,536 cycles.

```
T0_RELO  EQU   $C060
T0_RELH  EQU   $C061
T0_CTRL  EQU   $C062

; Fire roughly every 1 ms: 3,144,000 / 1000 = 3144 cycles = $0C48
         LDA   #$48
         STA   T0_RELO
         LDA   #$0C
         STA   T0_RELH
         LDA   #%00000011    ; enable + auto-reload
         STA   T0_CTRL
```

WHY REACH FOR A TIMER INSTEAD OF VBLANK

VBlank gives you exactly one tick per 60 Hz frame — fine for animation and input, too coarse for a metronome-accurate rhythm game, a UART-like protocol, or any effect that needs to subdivide a frame precisely. A hardware timer's IRQ fires on its own schedule, completely decoupled from the raster, and is the right tool whenever "once per frame" is the wrong granularity.

With controllers, the IRQ controller, and both timers under control, you have every input and every clock a Fanticon game can see. Part VI turns to shipping: bank switching, packaging a finished cartridge, and making sure it runs at full speed.

CART 06

06

SHIPPING A GAME

Bank switching at scale, packaging a cartridge, writing cycle-accurate code, and testing it before release.

CH 19 BANK SWITCHING · CH 20 PACKAGING & SAVES · CH 21 OPTIMIZATION · CH 22 TESTING & SHIPPING

CHAPTER 19 · PART VI

Bank Switching & Memory Management

> Everything you have built so far fit inside fixed ROM. A complete game outgrows 16 KiB quickly — the bank window is how Fanticon scales up to 4 MiB of code and assets without ever widening the CPU's address bus.

ONE WINDOW, FOUR POOLS

Think of `$8000-$BFFF` as a single 16 KiB peephole, not four separate ones. `BANK_KIND` decides *which pool* of memory the peephole currently looks into — cartridge ROM, work RAM, video RAM, or save RAM. `BANK_NUMBER` decides *which 16 KiB slice* of that pool you are looking at. Nothing else about addressing changes: `LDA $8005` always means "the sixth byte of whatever is mapped right now" — ROM, RAM, VRAM, or save RAM, depending entirely on the two registers below.

`BANK_KIND`	Valid `BANK_NUMBER`	Window contains
`$00`	0–255	Cartridge ROM, up to 4 MiB
`$01`	0–3	Volatile work RAM, 64 KiB total
`$02`	0–2	Video RAM, 48 KiB total
`$03`	0–3	Battery-backed save RAM, up to 64 KiB

Every bank switch is the same two-register move, regardless of which pool you are headed for:

1. Write the pool you want to `BANK_KIND` (`$C000`) — one of the four values above.
2. Write which slice of that pool to `BANK_NUMBER` (`$C001`).
3. Address `$8000-$BFFF` normally. It now means whatever you just selected, until the next write to either register.

The same CPU address means a different physical byte depending only on what those two registers currently hold:

BANK_KIND	BANK_NUMBER	$8005 currently means
$00	5	Byte 5 of cartridge ROM bank 5
$01	0	Byte 5 of work RAM bank 0
$02	0	Byte 5 of video RAM bank 0
$03	1	Byte 5 of save RAM bank 1

The change takes effect after the register write's bus cycle completes, so the golden rule is simple: **switch banks while executing from main RAM or fixed ROM, never from the bank window itself** — you would be changing the ground out from under the instruction that is running.

```
bank number = offset / $4000
CPU address = $8000 + (offset AND $3FFF)
```

Bank-aware symbols

Every label the assembler resolves records its CPU address *and* which section produced it. `BANKOF(label)` turns a switchable-ROM label into its bank number — it is an error to call it on a fixed-ROM, RAM, or plain numeric label:

```
        LDA   #BANKOF(LEVEL2)
        STA   BANKNUM
        JMP   LEVEL2      ; resolves to its $8000-$BFFF address
```

Referring to a banked label always returns its `$8000–$BFFF` address, and never silently inserts a bank switch for you — a direct branch or jump between two *different* switchable banks produces a build warning, because the assembler cannot know whether the correct bank will be mapped in when execution gets there.

What goes where

Work RAM (`BANK_KIND=$01`) is volatile banked scratch space — a natural home for level data staged out of ROM, decompression buffers, or music driver state that does not need to persist. Remember that only the lower 32 KiB of main RAM is *always* visible; work RAM banks exist purely inside the window. Save RAM (`$03`) is the one banked kind that survives power-off, covered fully in Chapter 20. A cartridge that declares fewer than four save banks simply exposes its unused bank numbers as unmapped.

A complete worked example: staging ROM into VRAM

Cartridge ROM and video RAM are two different pools behind the *same* window, so you can never have both mapped in at once — there is no register write that lets the CPU read ROM and write VRAM in the same instruction. Moving a level's graphics from ROM to VRAM is therefore always a three-step relay: map the ROM bank in and copy it out to ordinary main RAM, then map the VRAM bank in and copy from RAM into it. A small reusable routine handles the copying half of that relay regardless of which pool is mapped:

```
BANKKIND EQU   $C000
BANKNUM  EQU   $C001
SRC      EQU   $10      ; 2-byte zero-page pointer
DST      EQU   $12      ; 2-byte zero-page pointer
PAGES    EQU   $14      ; pages left to copy

; Copies A 256-byte pages from (SRC) to (DST). Load both pointers and A
; before calling; this only walks them forward one page at a time.
PAGECOPY STA   PAGES
.PAGE    LDY   #0
.BYTE    LDA   (SRC),Y
         STA   (DST),Y
         INY
         BNE   .BYTE
         INC   SRC+1
         INC   DST+1
         DEC   PAGES
         BNE   .PAGE
         RTS
```

With that in place, staging one level's 2,048-byte tile-number map from a switchable cartridge bank into video RAM is two calls — one bank switch and one `PAGECOPY` per leg of the relay:

```
        ; leg 1: cartridge ROM -> main RAM
        LDA   #$00
        STA   BANKKIND      ; cartridge ROM
        LDA   #BANKOF(LEVEL1_MAP)
        STA   BANKNUM       ; LEVEL1_MAP's bank now fills $8000-$BFFF

        LDA   #<LEVEL1_MAP
        STA   SRC
        LDA   #>LEVEL1_MAP
        STA   SRC+1
        LDA   #<STAGE
        STA   DST
        LDA   #>STAGE
        STA   DST+1
        LDA   #8            ; 2,048 bytes = 8 pages
        JSR   PAGECOPY

        ; leg 2: main RAM -> video RAM
        LDA   #$02
        STA   BANKKIND      ; video RAM
        LDA   #$00
        STA   BANKNUM       ; the tile-number map lives in VRAM bank 0

        LDA   #<STAGE
        STA   SRC
        LDA   #>STAGE
        STA   SRC+1
        LDA   #<$A000     ; $8000 + $2000, the tile-map offset from Ch. 9
        STA   DST
        LDA   #>$A000
        STA   DST+1
        LDA   #8
        JSR   PAGECOPY
```

`STAGE` is any 2,048-byte buffer you set aside in main RAM — it never needs to be bigger than the largest single asset you stage through it, and the same buffer is reused for tile patterns, attributes, sprite records, and bitmap pages by simply changing the source label, the destination offset, and the page count passed to `PAGECOPY`. Chapter 9 puts this exact relay to work loading a tile set.

RESET ALWAYS SELECTS BANK 0

Every reset selects cartridge bank 0, kind `$00`. Reset code cannot assume any other switchable bank is mapped — if your reset routine needs data from a different bank, it must explicitly select that bank itself before reading it, exactly like any other routine would.

With multiple banks in play, packaging becomes the next question: how does a project with a handful of referenced banks turn into one `.FCN` file, and how does a game remember player progress between sessions? Chapter 20 covers both.

CHAPTER 20 · PART VI

Packaging & Save Files

> A finished cartridge is a single `.FCN` file: a 64-byte header, a fixed 16 KiB ROM image, and zero or more banked ROM images, all little-endian and independently versioned from its optional `.SAV` companion.

How BUILD packs a cartridge

The packager writes the fixed image, then every bank from zero through the **highest bank your source references** — skipped banks and unwritten bytes fill with `$FF`, and any trailing banks above the highest reference are omitted entirely. Using banks 0 and 3, for instance, stores four banks, not 256 — you only pay for what you use.

Offset	Field	Meaning
`$00`	`MAGIC`	ASCII `FANTICON`
`$08/$09`	`FORMAT_MAJOR/MINOR`	`1.0`
`$0C`	`FLAGS`	Bit 0: uses battery RAM
`$10`	`ROM_BANKS`	Switchable banks, 0–256
`$12`	`SAVE_BANKS`	Battery RAM banks, 0–4
`$14/$18`	`FIXED_CRC32/BANKED_CRC32`	CRC-32 of ROM contents
`$1C`	`CARTRIDGE_ID`	Stable, nonzero 64-bit identity
`$24`	`TITLE`	22 bytes, printable ASCII
`$3A/$3B`	`MACHINE_MAJOR/MINOR`	Required hardware version

All CRC fields use CRC-32/ISO-HDLC. A loader validates, in strict order: minimum length and magic, supported version and header size, known flags and mapper, ROM/save bank-count limits, a nonzero ID with valid title bytes and compatible machine version, exact total file size, every checksum, and finally the presence of all three fixed-ROM vectors — before a single byte is exposed to the CPU. Nothing partially loads.

Save files

A cartridge with `SAVE_BANKS > 0` gets a sibling save file: on native platforms, the same base name with a `.SAV` extension beside the cartridge (`ASTRO.FCN` → `ASTRO.SAV`), keyed by path, not by the cartridge's internal title. Browser builds store the equivalent bytes in persistent browser storage, keyed by `CARTRIDGE_ID` instead.

Writes through the save-RAM bank window update RAM immediately and mark the save dirty. The host flushes it to disk after one second of no new writes, whenever a cartridge is cleanly ejected or Game mode exits, and during orderly shutdown — always through a temporary file that is flushed and then atomically swapped in, so a crash mid-save can only lose the most recent unflushed writes, never corrupt the last good file. Only one running Fanticon instance can hold the write lock for a given save; if a second instance cannot acquire it, that cartridge simply runs with save RAM read-only and a host warning.

WHY CARTRIDGE_ID MATTERS

`ID` in `FANTICON.CFG` is generated once, the first time a project is created, and ordinary rebuilds never change it — that is deliberate. It is what lets you ship new versions of a game, with completely different code and assets, while every player's existing save file keeps working. If a loaded save's declared size does not match `SAVE_BANKS × $4000` for the cartridge that is loading it, Fanticon deletes the mismatched save and creates a fresh zero-filled one at the correct size — save data is never silently migrated across a size change.

Packaging and saving are both automatic once your manifest and sections are right — the work that is yours to get right is making the code fast enough to hit 60 Hz every frame, which is where Chapter 21 goes next.

CHAPTER 21 · PART VI

Cycle-Accurate Optimization

> Fanticon gives you a fixed budget every frame: 52,400 CPU cycles at 60 Hz. Writing efficient 6502 code is mostly about arranging data around the processor, not chasing clever instruction tricks.

- Put frequently accessed variables and pointers in **zero page** — a zero-page load is one byte shorter and one cycle faster than the equivalent absolute load.
- Use `X` or `Y` as loop counters so `DEX`/`DEY`/`INX`/`INY` avoid memory traffic entirely.
- Choose `(zp),Y` for walking one buffer through a zero-page pointer, and `(zp,X)` for selecting among several adjacent zero-page pointers.
- Align tables to page boundaries when indexed-read timing must stay constant — an `abs,X` or `abs,Y` read costs one extra cycle whenever the index crosses a page.
- Keep hot loops inside a single page so a taken branch never pays the page-crossing penalty, and remember the final (not-taken) iteration of a counted loop usually has a different cycle cost than the taken ones.
- Prefer a counted loop over unrolled code until you have measured a bottleneck — unroll only after profiling shows it matters.

```
; X iterations: 5 cycles per taken pass, 4 on the final (not-taken) pass
LOOP    DEX             ; 2 cycles
        BNE   LOOP        ; 3 taken, 2 not taken (same-page target)
```

HARDWARE REGISTERS ARE NOT ORDINARY MEMORY

Fanticon exposes exact NMOS bus timing: indexed stores always perform a dummy read before the write, and a read-modify-write instruction (`INC` , `ASL` , and friends) reads the old value, writes it back unchanged, and only then writes the new value — three bus accesses where you might expect one. Never use a read-modify-write instruction on a device register unless that register's own documentation explicitly says it tolerates the extra read and write. The same caution applies to indexed addressing near a device register: an indexed access can briefly touch a provisional address on its way to the final one.

A portability and correctness checklist

Before treating any routine as finished:

- Know exactly which registers, flags, zero-page bytes, and stack bytes it clobbers.
- Set carry explicitly (`CLC`/`SEC`) before every independent `ADC`/`SBC` chain — never assume its state.
- Confirm decimal mode is what you expect; NMOS interrupt entry does not clear `D` for you.
- Check every relative branch is in range, and whether a page crossing changes its timing.
- Never assume RESET initializes `A`, `X`, or `Y` — initialize your own program state explicitly, every time.
- Keep `SP` balanced across every control-flow path, and end interrupt handlers with `RTI`, ordinary subroutines with `RTS` — never mix the two.
- Avoid undocumented opcodes in anything you might someday retarget to a different 6502-family chip; Fanticon emulates all 256 opcode bytes faithfully, but that faithfulness is exactly why relying on them is a one-way door.

Appendix A collects the complete opcode and cycle tables this chapter draws its rules of thumb from, including the full undocumented-opcode catalog for when you deliberately decide the trade-off is worth it. Chapter 22 closes the loop: how Fanticon itself verifies correctness, and how to give your own cartridge the same confidence before it ships.

CHAPTER 22 · PART VI

Testing & Shipping

> Fanticon itself is held to an exacting standard: cycle-accurate, bit-identical behavior on every host, every time. Here is how to borrow that same confidence for your own game before you call it done.

Playtesting with the debugger, deliberately

Chapter 6 introduced breakpoints, watchpoints, and stepping as tools for understanding code you just wrote. Before shipping, turn them on your *finished* cartridge instead:

- Set a memory-write watchpoint on your player's health or position variable and play normally — anything that writes it unexpectedly stands out immediately.
- Drop a breakpoint at the top of your VBlank handler and single-step (`F11`) through one full frame at least once, confirming every interrupt source you expect to see acknowledged is.
- Use a raster breakpoint at the exact `LINE,DOT` of a raster effect (Chapter 12) to confirm it fires where you think it does, not just that it looks right on screen.
- Watch the paused panel's APU section while audio plays to confirm channels enable, retrigger, and disable exactly when your code intends.

Budget every frame, not just the average one

52,400 cycles is the hard limit, but the moment that matters is your *worst* frame, not your typical one — the frame where the player is taking damage, three enemies are on screen, a room transition is loading, and the tracker is mid-pattern all at once. Trigger that worst case deliberately while stepping through it, and confirm your VBlank handler still finishes before the next VBlank arrives. A handler that occasionally overruns shows up as a dropped frame or a one-frame input hiccup — often intermittent enough to ship unnoticed unless you have gone looking for it on purpose.

Exercise every save path

Chapter 20 covered how save RAM persists; test all three shapes that behavior can take before release:

- A completely fresh save, from a player who has never run your cartridge before.

- A save created by an earlier build of your own cartridge, loaded by the current one.
- What happens if `SAVE_BANKS` in your manifest ever changes between versions — Fanticon discards a save whose size no longer matches and starts over rather than guessing, so make sure that is an acceptable experience rather than a surprise.

A pre-ship checklist for your own cartridge

- Confirm your frame's work fits inside 52,400 cycles with room to spare for worst-case input/animation/audio bursts.
- Step through RESET and your interrupt handlers in the debugger at least once end to end; confirm every acknowledgment clears the bit it means to.
- If Fanticon is available to you on more than one platform (say, native and browser), play your finished cartridge on both — save persistence in particular takes a different path on each, as Chapter 20 described.
- Re-read the correctness checklist at the end of Chapter 21 against your source, not from memory.

That is the complete path from an empty folder to a shipped cartridge. Part VII turns from the console itself to the games you build with it — menus, side-scrollers, top-down worlds, raster effects, and palette tricks, all in genre-shaped detail. The appendices after that are the reference material you will keep this book open to.

CART 07

07

GAME DEV PATTERNS

Turning the raw hardware from Parts II–VI into menus, side-scrollers, top-down worlds, raster tricks, and palette-swap magic.

CH 23 LOOPS, STATES & MENUS · CH 24 SIDE-SCROLLERS · CH 25 TOP-DOWN & RPG · CH 26 RASTER EFFECTS · CH 27 PALETTE SWAPPING

CHAPTER 23 · PART VII

Game Loops, States & Building a Main Menu

> Almost every Fanticon game, regardless of genre, is structured the same way underneath: a VBlank-driven heartbeat, and a state machine deciding what that heartbeat does this frame. A main menu is simply the first state.

The heartbeat

Chapter 17 built a VBlank handler that reads input, updates the game, and ticks music. Keep that structure for the life of the project; what changes screen to screen is only what `UPDATEGAME` does. The cleanest way to express "what it does depends on where the player currently is" is a small state variable and a jump table:

```
GAMESTATE EQU  $40    ; 0=menu, 1=play, 2=pause
STATEPTR  EQU  $41    ; 2-byte zero-page pointer

UPDATEGAME
          LDX   GAMESTATE
          LDA   STATE_LO,X
          STA   STATEPTR
          LDA   STATE_HI,X
          STA   STATEPTR+1
          JMP   (STATEPTR)  ; the 6502's one indirect jump form

STATE_LO  DFB   <MENU_UPDATE,<PLAY_UPDATE,<PAUSE_UPDATE
STATE_HI  DFB   >MENU_UPDATE,>PLAY_UPDATE,>PAUSE_UPDATE
```

This is the standard 6502 jump-table idiom: since `JMP` only has one indirect form (a plain pointer, not an indexed one), you compute the target address into a zero-page pointer yourself using the low/high byte operators `<` and `>` from Chapter 4, then jump through it. Adding a new state is one entry in two tables, not a growing chain of `CMP`/`BEQ` checks.

Building the menu screen itself

A main menu is ordinary tile-mode content: a static background built once in the graphics editor (Chapter 8), with menu text authored directly as tiles rather than drawn at runtime. Selection typically works one of two ways:

- **Attribute swap** — give the currently selected menu row's tiles a different palette bank in the attribute map (Chapter 9), so "highlighting" an option is one small VRAM write, not a redraw.
- **Cursor sprite** — park one 8×8 sprite next to the selected row and move its `Y` field to point at a new row on Up/Down.

```
MENUSEL  EQU  $42    ; 0-2: which menu row is selected

MENU_UPDATE
         LDA   PAD0P
         AND   #BTN_DOWN
         BEQ   CHECKUP
         INC   MENUSEL
         LDA   MENUSEL
         CMP   #3
         BNE   MOVED
         LDA   #0
         STA   MENUSEL     ; wrap back to the top entry
MOVED    JSR   MOVECURSOR
CHECKUP
         ; ...BTN_UP handled the same way, decrementing...
         LDA   PAD0P
         AND   #BTN_A
         BEQ   MENUDONE
         LDA   #1
         STA   GAMESTATE   ; confirm -- switch to the PLAY state
MENUDONE
         RTS
```

Transitioning between screens

Because a state change is just writing `GAMESTATE`, the interesting work is usually what happens once, on the frame a transition starts: swapping in the gameplay `.GFX` set (Chapter 9), selecting the right cartridge bank if the new screen's code lives elsewhere (Chapter 19), and resetting per-screen RAM. Do that setup work in a dedicated `ENTER_PLAY` routine called exactly once on the transition, not inside `PLAY_UPDATE` itself — otherwise it silently re-runs every frame you are in that state.

PAUSE IS JUST ANOTHER STATE

Because the state machine already exists, pause is nearly free: a `PAUSE_UPDATE` handler that ignores gameplay input, watches only for the pause button to return to state 1, and optionally stops calling `MUSIC_TICK` (Chapter 15) so the soundtrack freezes with everything else.

With state management in place, the next three chapters are about what *lives* inside the "play" state for three different kinds of games.

CHAPTER 24 · PART VII

Side-Scrolling Games

> A side-scroller is Chapter 9's ring-buffer scrolling technique, plus a player that moves smoothly through it and a world that pushes back.

Sub-pixel movement

Smooth in-between speeds come from storing position as a 16-bit fixed-point value, high byte the pixel position and low byte a sub-pixel fraction, and adding a small velocity to the whole 16-bit value every frame:

```
PLAYERX  EQU  $50    ; sub-pixel fraction
PLAYERX_HI EQU $51 ; whole pixel position
VELX     EQU  $52    ; e.g. $180 = 1.5 px/frame

         CLC
         LDA   PLAYERX
         ADC   VELX
         STA   PLAYERX
         LDA   PLAYERX_HI
         ADC   VELX+1
         STA   PLAYERX_HI  ; carry propagates into the pixel byte automatically
```

Only `PLAYERX_HI` ever needs to reach the sprite record's `X_LOW` byte (Chapter 10); the fraction stays entirely in RAM. This is exactly how the wave demo's smooth raster curve and countless 8-bit platformers get sub-integer motion out of an integer CPU.

Collision is a lookup table

Tile collision is software work: reserve a range of tile numbers for solid terrain and check the map, not the screen:

```
; Is the tile under world pixel (X,Y) solid? Tiles $00-$1F = passable, $20+ =
solid.
ISSOLID
         ; ...compute tile_x = X/8, tile_y = Y/8, read the map byte at
$2000+tile_y*64+tile_x...
         LDA   #$20
         CMP   MAPBYTE
         RTS            ; carry clear if MAPBYTE < $20 (passable)
```

Checking a handful of sample points around the player's bounding box (feet, head, both sides) against this table is enough for a solid platformer — the same approach scales down to one point for a simple top-down game in Chapter 25.

Following the player with the camera

Write `SCROLL_X` from a camera variable, not directly from the player's position — a camera that is always centered exactly on the player feels jittery and reveals level edges the moment the player stops near them. A small deadzone keeps the camera still while the player moves within the middle of the screen, then catches up smoothly, and clamping keeps it from scrolling past the level's edges:

```
        ; if player is left of camera+deadzone, pull camera left; mirror on the
right
        ; then clamp: if camera < 0, camera = 0; if camera > level_max, camera =
level_max
```

Streaming new columns without blowing the frame budget

Chapter 9 described the ring-buffer idea: as the camera crosses an 8-pixel tile boundary, the column of map cells about to scroll into view needs fresh tile numbers written before it arrives. Writing an entire 32-cell column in one VBlank is usually fine, but if a level streams from compressed or procedurally generated data, cap how much work happens per frame (a handful of cells) and spread the rest across the next few frames — the hidden margin outside the 320×200 viewport (Chapter 9) is exactly the slack that makes this possible.

> **FAKING PARALLAX ON ONE LAYER**
>
> Convincing depth on single-layer 8-bit hardware usually comes from restraint: a static or slow backdrop color change instead of a moving backdrop, or a few large sprites drifting at a different speed than the world for a lightweight parallax illusion, budgeted against the eight-sprites-per-line limit from Chapter 10.

CHAPTER 25 · PART VII

Top-Down & RPG-Style Games

> Top-down games trade continuous scrolling for grid discipline: characters move cell by cell, and worlds bigger than one screen are built as connected rooms rather than one continuously scrolling map.

Grid-locked movement

Rather than free sub-pixel motion, classic top-down movement snaps to an 8 or 16-pixel grid matching the tile size, animating *between* cells over a fixed number of frames:

```
MOVETIMER EQU $53   ; counts down while sliding between cells
MOVEDX    EQU $54    ; +1, 0, or -1 this move

TRYMOVE
          LDA   MOVETIMER
          BNE   BUSY        ; already sliding -- ignore new input
          ; ...check the destination cell with the same ISSOLID-style lookup as
Chapter 24...
          LDA   #8
          STA   MOVETIMER   ; 8 frames to cross one 8px cell = 1px/frame
BUSY      RTS
```

Each frame that `MOVETIMER` is nonzero, add `MOVEDX`/`MOVEDY` to the player's pixel position and decrement the timer; input is only sampled again once it reaches zero. This single rule — a move either fully completes or has not started — is what makes grid movement feel precise instead of mushy, and it makes tile collision trivial: you only ever need to validate the single destination cell before committing to a move, never a mid-slide position.

Worlds bigger than one map

The hardware tilemap is a fixed 64×32 cells — plenty for one scrolling side-scrolling level, but an RPG overworld is usually built from many discrete **rooms**, each its own `.GFX` map, rather than one giant scrolling space. When the player walks off the edge of the current room, load the next one using the same VRAM-loading routine from Chapter 9 — select the neighboring room's bank (Chapter 19), copy its patterns/map/attributes into VRAM, and place the player at the matching entry point on the new map's opposite edge. Unlike side-scrolling, this is a deliberate hard cut rather than continuous motion, so it reads well with a brief screen wipe (Chapter 26) covering the one or two frames the load takes.

NPCs, facing, and interaction

An NPC is a sprite plus a small record: grid position, facing direction, and a pointer to whatever it does when interacted with. "Talk to the NPC in front of you" is then just: compute the cell one step ahead of the player in their current facing direction, and check it against the NPC table the same way `ISSOLID` checks it against terrain:

```
; FACEDX/FACEDY are -1/0/+1 based on the player's last-pressed direction
TRYINTERACT
         ; targetcell = playercell + (FACEDX, FACEDY)
         ; ...scan the NPC table for an entry at targetcell...
         ; if found: JSR through that NPC's dialogue pointer
```

Text and dialogue boxes

Because tile-map cells can hold any of the 256 resident patterns, author a text tileset (one 8×8 pattern per glyph) in the graphics editor and reserve a palette bank for it (Chapter 8). "Printing" a message is then writing tile numbers directly into a fixed region of the tile-number map — usually a few rows near the bottom of the screen — one character (one `STA`) at a time, optionally paced a letter every few frames for a classic scrolling-text effect. Because this is ordinary VRAM, the same dialogue box works whether it appears over a top-down room or a side-scrolling level.

BUFFER THE INPUT, NOT JUST THE POSITION

A common RPG feel comes from accepting a directional press slightly before the current move finishes and queuing it, rather than dropping it because `MOVETIMER` was still nonzero. Latch the most recent direction pressed into a one-entry buffer in your VBlank handler, and consult that buffer instead of a fresh read the instant `MOVETIMER` reaches zero — it is a small change that makes continuous movement across several cells feel responsive instead of stuttery.

CHAPTER 26 · PART VII

Raster Effects in Practice

> Chapter 12 covered the raster comparator itself. This chapter applies it to three effects nearly every genre eventually wants: a status bar that will not scroll, animated color cycling, and a screen wipe between scenes.

A non-scrolling status bar via a scroll split

Fanticon has one tile layer, so a HUD that stays put while the world above it scrolls needs a trick, not a second layer. Reserve the bottom rows of your 64×32 map (say, rows 30–31) exclusively for HUD tiles, keep your world scroll's *vertical* range clamped so those rows never scroll into view during normal play, and then use a raster IRQ at the exact scanline where the HUD begins to rewrite `SCROLL_Y` so the remaining visible lines display that fixed HUD region instead of continuing the world:

```
; Raster IRQ fires at line 184 -- the last 16 lines become the HUD
HUDSPLIT
         LDA   #$00
         STA   SCROLLY_LO
         LDA   #240      ; points SCROLL_Y at map row 30 (240 / 8)
         STA   SCROLLY_HI  ; (illustrative split -- tune to your HUD's row count)
         ; re-arm RASTER_Y for line 0 next frame, then restore the world SCROLL_Y
there
```

A second raster target back at line 0 (or simply doing it at the top of your next VBlank handler) restores the camera scroll before the world begins drawing again next frame. This is the same family of technique NES games used for split-screen status bars — here it is simpler, because Chapter 7's dual-ported VRAM means the write always lands exactly on the raster timestamp you programmed, with no bus contention to fight.

Palette cycling

Animating water, lava, or energy fields without touching a single tile is a palette trick: rotate a small run of entries in one bank every few frames.

```
CYCLE   LDA   COLOR3      ; save the entry about to be overwritten
        PHA
        LDA   COLOR2
        STA   COLOR3
        LDA   COLOR1
        STA   COLOR2
        PLA
        STA   COLOR1      ; write all three back through PALETTE_INDEX/DATA
```

Because every tile using that palette bank updates the instant the palette entry changes (Chapter 7), a whole river or lava field animates from one small per-frame write, regardless of how many tiles are on screen.

Screen wipes between scenes

The cheapest transition is a straight cut. The next cheapest, and usually worth it, fades through `BACKDROP_COLOR` with the background layer momentarily disabled (`VIDEO_CONTROL` bit 0 clear, Chapter 7) — step the backdrop through a short table of darkening RGB332 values over a handful of frames, do your room load (Chapter 25) while the screen reads as solid black, then step back up and re-enable the background. For a full-color wipe instead of fading to black, step every entry of the active palette bank(s) toward its target using the same per-frame table idea from palette cycling above.

All three of these effects are variations on one idea: Fanticon guarantees a write lands exactly where you time it, so "when" is as much a creative tool here as "what."

CHAPTER 27 · PART VII

Palette Swapping & Color Tricks

> Sixteen palette banks sharing one pattern library (Chapter 7) is one of the cheapest tools in the whole machine: the same artwork, a completely different look, for the cost of a register write.

Recoloring for free

An enemy palette swap is not a second set of sprite patterns — it is the same tile numbers with a different palette bank in the sprite's `ATTR` byte (Chapter 10) or the tile attribute map (Chapter 9). A "red slime" and a "blue slime" can be one pattern, two palette banks, and zero extra VRAM for pixels:

```
        LDA   #%10000001 ; enabled, 8x8, palette bank 1 (red variant)
        STA   REC0+4
        ; ...later, for the blue variant, only the low nibble changes...
        LDA   #%10000011 ; same sprite, palette bank 3 (blue variant)
        STA   REC1+4
```

Damage flash and invincibility blink

A hit-flash is a palette bank swap held for a handful of frames — write the sprite's palette bits to a bank containing an all-white or all-red version of the same ramp for a few frames, then restore the normal bank. Invincibility blinking is the same idea driven by a timer in your VBlank handler, alternately toggling the sprite's `ENABLE` bit (or swapping between the normal bank and a dimmed one) every few frames until the timer expires — both are RAM-side bookkeeping around registers already covered in Chapter 10, not new hardware capability.

Re-tinting a whole scene

Because `PALETTE_INDEX`/`PALETTE_DATA` can rewrite any of the 256 entries at any time (Chapter 7), a day-to-night transition, a "poisoned" status tint, or an alternate title-screen look for a bitmap (via `BITMAP_PALETTE`, Chapter 11) can all be authored as a handful of extra palette banks in the graphics editor and swapped in with the same loop from Chapter 8's palette-upload example:

```
PALINDEX EQU   $C01B
PALDATA  EQU   $C01C

         LDA   #$30       ; bank 3, entry 0 -- the "level" bank from Ch. 8's table
         STA   PALINDEX
         LDX   #0
RETINT   LDA   NIGHT_PAL,X
         STA   PALDATA     ; index auto-increments -- 16 writes recolor the whole
bank
         INX
         CPX   #16
         BNE   RETINT
```

Sixteen writes re-skins every tile and sprite already using that bank, instantly, with no changes to map data, pattern data, or code anywhere else — a large visual change for one of the smallest possible pieces of work on the machine.

RESERVE A BANK ON PURPOSE

Chapter 8's suggested bank layout (`UI`, `player`, `enemies`, `level`, `title`) is worth extending deliberately: if you know a scene will need a flash, a tint, or a variant color scheme, reserve its palette bank up front rather than discovering late that every other bank is already spoken for. There are only sixteen.

That closes the loop from raw hardware to genre-shaped technique. Whatever kind of game you are building, it is assembled from exactly the pieces in Parts II–VII — the appendices that follow are where you go to check the exact register, cycle count, or byte layout behind any of them.

REFERENCE

R

APPENDICES

The exact tables and formats you will keep this book open to — opcode timings, the full memory map, the assembler, both file formats, every demo cartridge, and a glossary.

APP A CPU OPCODES · APP B MEMORY MAP · APP C ASSEMBLER · APP D FILE FORMATS · APP E DEMOS · APP F GLOSSARY

APPENDIX A

Complete 6502 Opcode & Cycle Reference

> Fanticon implements the standard NMOS MOS 6502 — not the CMOS 65C02 — including every undocumented opcode byte. This appendix is the exact reference; the teaching version is Chapter 3.

Official opcode and cycle table

Each cell is `opcode/cycles`. `+P` means one additional cycle on a page crossing. A dash means the instruction has no such addressing mode.

Instr	#	zp	zp,X	zp,Y	abs	abs,X	abs,Y	(zp,X)	(zp),Y
ADC	69/2	65/3	75/4	—	6D/4	7D/4+P	79/4+P	61/6	71/5+P
AND	29/2	25/3	35/4	—	2D/4	3D/4+P	39/4+P	21/6	31/5+P
CMP	C9/2	C5/3	D5/4	—	CD/4	DD/4+P	D9/4+P	C1/6	D1/5+P
EOR	49/2	45/3	55/4	—	4D/4	5D/4+P	59/4+P	41/6	51/5+P
LDA	A9/2	A5/3	B5/4	—	AD/4	BD/4+P	B9/4+P	A1/6	B1/5+P
ORA	09/2	05/3	15/4	—	0D/4	1D/4+P	19/4+P	01/6	11/5+P
SBC	E9/2	E5/3	F5/4	—	ED/4	FD/4+P	F9/4+P	E1/6	F1/5+P
LDX	A2/2	A6/3	—	B6/4	AE/4	—	BE/4+P	—	—
LDY	A0/2	A4/3	B4/4	—	AC/4	BC/4+P	—	—	—
CPX	E0/2	E4/3	—	—	EC/4	—	—	—	—
CPY	C0/2	C4/3	—	—	CC/4	—	—	—	—
STA	—	85/3	95/4	—	8D/4	9D/5	99/5	81/6	91/6
STX	—	86/3	—	96/4	8E/4	—	—	—	—
STY	—	84/3	94/4	—	8C/4	—	—	—	—

Read-modify-write instructions (fixed timing)

Instr	A	zp	zp,X	abs	abs,X
ASL	0A/2	06/5	16/6	0E/6	1E/7
LSR	4A/2	46/5	56/6	4E/6	5E/7
ROL	2A/2	26/5	36/6	2E/6	3E/7
ROR	6A/2	66/5	76/6	6E/6	7E/7
DEC	—	C6/5	D6/6	CE/6	DE/7

Instr	A	zp	zp,X	abs	abs,X
INC	—	E6/5	F6/6	EE/6	FE/7

A read-modify-write instruction reads the value, writes the old value back unchanged, then writes the new value — three bus accesses, not one. See the hardware-register warning in Chapter 21.

Other official opcodes

Group	Opcodes and cycles
Branch	`BPL 10`, `BMI 30`, `BVC 50`, `BVS 70`, `BCC 90`, `BCS B0`, `BNE D0`, `BEQ F0` — all 2 + taken + page
Bit test	`BIT zp 24/3`, `BIT abs 2C/4`
Jump/call	`JMP abs 4C/3`, `JMP (abs) 6C/5`, `JSR 20/6`, `RTS 60/6`, `RTI 40/6`
Stack	`PHP 08/3`, `PLP 28/4`, `PHA 48/3`, `PLA 68/4`
Transfer	`DEY 88/2`, `TXA 8A/2`, `TAY A8/2`, `TAX AA/2`, `TYA 98/2`, `TXS 9A/2`, `TSX BA/2`
Increment	`INY C8/2`, `INX E8/2`, `DEX CA/2`
Flags	`CLC 18/2`, `SEC 38/2`, `CLI 58/2`, `SEI 78/2`, `CLV B8/2`, `CLD D8/2`, `SED F8/2`
System	`BRK 00/7`, `NOP EA/2`

Cycle-accurate programming rules

- Taken branches perform an extra read; a page-crossing branch performs another.
- Indexed reads perform a correction cycle when their address crosses a page.
- Indexed stores always perform a dummy read before the write.
- Stack and control-flow instructions perform their documented dummy reads.
- Holding the hardware-ready signal inactive repeats read cycles but never stalls a write.

Decimal mode

With `D=1`, `ADC`/`SBC` perform NMOS binary-coded-decimal adjustment — each nibble is a decimal digit, so `$42` means decimal 42. Fanticon implements NMOS decimal behavior exactly, including inputs that are not valid BCD digits. `N`, `V`, and `Z` after a decimal operation are NMOS-specific and should not be read as portable BCD indicators — prefer `C` for carry/no-borrow and test `A` directly when it matters. Interrupt entry does not clear `D`; leave decimal mode clear except inside a small, deliberately controlled BCD routine.

Undocumented NMOS opcodes

Fanticon's assembler accepts every stable undocumented mnemonic below. Where the NMOS has duplicate opcode bytes for identical syntax, the assembler emits one canonical encoding (`$0B` for `ANC #`, `$02` for `KIL`/`JAM`, and `$80`/`$04`/`$14`/`$0C`/`$1C` for addressed `NOP`); use `DFB`/`HEX` to force a specific alternate byte.

Mnemonic	Operation	Opcodes by mode (same order as official table)
SLO	`ASL M`, then `A←A|M`	03,07,0F,13,17,1B,1F
RLA	`ROL M`, then `A←A&M`	23,27,2F,33,37,3B,3F
SRE	`LSR M`, then `A←A^M`	43,47,4F,53,57,5B,5F
RRA	`ROR M`, then `ADC M`	63,67,6F,73,77,7B,7F
SAX	`M←A&X`	(zp,X)83, zp87, abs8F, zp,Y97
LAX	`A←M; X←M`	(zp,X)A3, zpA7, absAF, (zp),YB3, zp,YB7, abs,YBF
DCP	`DEC M`, then compare A with M	C3,C7,CF,D3,D7,DB,DF
ISC/ISB	`INC M`, then `SBC M`	E3,E7,EF,F3,F7,FB,FF

For the seven-mode rows, cycle counts in listed order are 8, 5, 6, 8, 6, 7, 7 — these are read-modify-write operations that perform both the old-value and new-value writes.

Mnemonic	Opcode	Cyc	Operation
ANC #	0B, 2B	2	`A←A&#` ; C←result bit 7
ALR #	4B	2	`A←A&#`, then LSR
ARR #	6B	2	`A←A&#`, then ROR; special C/V/decimal behavior
XAA #	8B	2	`A←(A|$EE)&X&#`
LAX #	AB	2	`A←X←(A|$EE)&#`
AXS/SBX #	CB	2	`X←(A&X)-#`; C reports no borrow
SBC #	EB	2	Alias of official immediate SBC
LAS abs,Y	BB	4+P	`A←X←SP←M&SP`
AHX (zp),Y / abs,Y	93 / 9F	6 / 5	Unstable high-byte-masked store
TAS abs,Y	9B	5	`SP←A&X`, then unstable masked store
SHY abs,X	9C	5	Store Y masked by address-derived high byte
SHX abs,Y	9E	5	Store X masked by address-derived high byte

The unstable stores follow Fanticon's own fixed, deterministic behavior exactly (including its `$EE` magic-mask value), which differs from how these bytes behave on an original chip, where results vary with silicon revision, temperature, and bus conditions.

Undocumented NOP form	Opcodes	Cycles
Implied	1A 3A 5A 7A DA FA	2
Immediate	80 82 89 C2 E2	2
Zero page	04 44 64	3
Zero page,X	14 34 54 74 D4 F4	4
Absolute	0C	4
Absolute,X	1C 3C 5C 7C DC FC	4+P

Undocumented NOPs still fetch operands and perform normal addressing-mode bus reads — they are not all equivalent to `EA`. The JAM/KIL opcodes (`02 12 22 32 42 52 62 72 92 B2 D2 F2`) halt useful execution until RESET; do not use JAM as a wait primitive unless that halt-until-reset behavior is exactly the intent.

APPENDIX B

Complete Memory Map

> Every address a Fanticon cartridge can see, in one place. All addresses are hexadecimal; multi-byte registers are little-endian.

CPU address space

Range	Size	Read	Write
`$0000–$7FFF`	32 KiB	Main RAM	Main RAM
`$8000–$BFFF`	16 KiB	Selected bank	Selected bank, unless ROM
`$C000–$C0FF`	256 B	Device registers	Device registers
`$C100–$FFFF`	16,128 B	Fixed cartridge ROM	Ignored

Unmapped and reserved reads return `$FF`; unmapped, reserved, and ROM writes are ignored outright — Fanticon does not emulate a floating last-value data bus.

Bank window ($8000–$BFFF)

Register	Address	Purpose
`BANK_KIND`	`$C000`	Select the backing memory type
`BANK_NUMBER`	`$C001`	Select one bank of that type

`BANK_KIND`	Valid `BANK_NUMBER`	Window contains
`$00`	0–255	Cartridge ROM, up to 4 MiB
`$01`	0–3	Volatile work RAM, 64 KiB total
`$02`	0–2	Video RAM, 48 KiB total
`$03`	0–3	Battery-backed save RAM, up to 64 KiB total
Other	—	Unmapped memory

```
bank number = offset / $4000
CPU address = $8000 + (offset AND $3FFF)
```

I/O page overview ($C000–$C0FF)

Address	Device
`$C000–$C00F`	Banking, interrupt controller, frame counter
`$C010–$C02F`	Video control and palette

`$C030–$C040`	Audio channels and master control
`$C050–$C05F`	Controllers
`$C060–$C06F`	Two interval timers
`$C070–$C0FF`	Reserved

System and interrupt registers

Addr	Name	Access	Description
$C000	BANK_KIND	R/W	$00 cartridge, $01 work RAM, $02 VRAM, $03 save RAM
$C001	BANK_NUMBER	R/W	Bank within the selected kind
$C002	IRQ_PENDING	R/W1C	Pending interrupt sources
$C003	IRQ_ENABLE	R/W	Sources allowed to assert IRQ
$C004	FRAME_LOW	R	Frame counter low byte; latches high byte
$C005	FRAME_HIGH	R	Latched frame counter high byte
$C006	MACHINE_MAJOR	R	Hardware major version, currently 1
$C007	MACHINE_MINOR	R	Hardware minor version, currently 0

Bit	Mask	Source
0	$01	Vertical blank begins
1	$02	Raster X/Y comparison matches
2	$04	Timer 0 reaches zero
3	$08	Timer 1 reaches zero

Video registers

Addr	Name	Access	Description
$C010	VIDEO_MODE	R/W	0 blank, 1 tilemap, 2 packed bitmap
$C011	VIDEO_CONTROL	R/W	Bit 0 background enable, bit 1 sprites enable
$C012	BACKDROP_COLOR	R/W	8-bit palette index behind a disabled background
$C013/$C014	SCROLL_X	R/W	Signed 16-bit tilemap X scroll
$C015/$C016	SCROLL_Y	R/W	Signed 16-bit tilemap Y scroll
$C017/$C018	RASTER_X	R/W	Raster compare dot, 0–399
$C019/$C01A	RASTER_Y	R/W	Raster compare line, 0–261
$C01B	PALETTE_INDEX	R/W	Select one of 256 palette entries
$C01C	PALETTE_DATA	R/W	RGB332 data; index auto-increments
$C01D	BITMAP_PALETTE	R/W	Bitmap palette bank, bits 0–3
$C01E	VIDEO_STATUS	R	VBlank, HBlank, sprite-overflow status — see below

Bit	VIDEO_STATUS field

0	Live VBlank
1	Live HBlank
2	Sprite overflow, latched for the current frame
7–3	Reserved

Video RAM — tile mode

VRAM offset	Bank	Size	Description
$0000–$1FFF	0	8 KiB	256 packed 4-bpp, 8×8 tile patterns
$2000–$27FF	0	2,048 B	64×32 tile-number map
$2800–$2FFF	0	2,048 B	64×32 tile-attribute map
$3000–$30FF	0	256 B	32 eight-byte sprite records

```
tile pattern offset = tile_number × 32
tilemap offset      = tile_y × 64 + tile_x
tile number byte    = $2000 + tilemap offset
tile attribute byte = $2800 + tilemap offset
sprite record       = $3000 + sprite_number × 8
```

Bit	Tile attribute field
7	Reserved
6	Priority
5	V-flip
4	H-flip
3–0	Palette

Sprite record (8 bytes)

Offset	Name	Meaning
0	X_LOW	X position bits 0–7
1	X_FLAGS	Bit 0 X bit 8; bit 1 behind background
2	Y	Y position
3	TILE	First pattern tile
4	ATTR	Enable, size, flips, palette — see below

Bit	ATTR field
7	Enable
6	16×16 size
5	V-flip
4	H-flip
3–0	Palette

X `$1F0-$1FF` = −16..−1. Y `$F0-$FF` = −16..−1. Sprites clip at all four edges and never wrap.

Video RAM — bitmap mode

VRAM offset	Bank	Size	Description
$4000–$7FFF	1	16 KiB	First bitmap section
$8000–$BCFF	2	15,616 B	Remaining bitmap pixels

```
VRAM byte offset = $4000 + Y × 160 + (X / 2)
even X pixel = high nibble   odd X pixel = low nibble
```

Audio registers

Channel	Control	Timer low	Timer high	Phase reset
Pulse 1	$C030	$C031	$C032	$C033
Pulse 2	$C034	$C035	$C036	$C037

Bit	Pulse control field
7	Enable
6–5	Duty
4	Reserved
3–0	Volume

Addr	Name	Description
$C038	TRI_CONTROL	Bit 7 enables the triangle
$C039/$C03A	TRI_TIMER	11-bit timer
$C03B	TRI_PHASE_RESET	Any write resets phase
$C03D	NOISE_PERIOD	Period-table index, bits 0–3
$C03E	NOISE_RESET	Any write reseeds the LFSR

Bit	NOISE_CONTROL field ($C03C)
7	Enable
6	Short mode
5–4	Reserved
3–0	Volume

Bit	AUDIO_MASTER field ($C040)
7	Master enable
6–4	Reserved
3–0	Master volume

Controller registers

Addr	Name	Description
$C050	PAD0_STATE	Currently held controller-1 buttons
$C051	PAD0_PRESSED	Newly pressed buttons; read clears returned bits
$C052	PAD1_STATE	Currently held controller-2 buttons
$C053	PAD1_PRESSED	Newly pressed buttons; read clears returned bits

```
7       6       5       4       3       2       1       0
START   SELECT  B       A       RIGHT   LEFT    DOWN    UP
```

Timer registers

Timer	Reload lo	Reload hi	Control	Count lo	Count hi
Timer 0	$C060	$C061	$C062	$C063	$C064
Timer 1	$C068	$C069	$C06A	$C06B	$C06C

Control: `$01` enable, `$02` auto-reload. Both timers decrement once per CPU cycle; reload of zero means 65,536 cycles.

CPU vectors

Address	Vector
$FFFA–$FFFB	NMI — reserved by v0.1 hardware
$FFFC–$FFFD	RESET entry point
$FFFE–$FFFF	IRQ and BRK entry point

Fixed ROM must supply all three vectors even when a cartridge never uses NMI or IRQ.

APPENDIX C

Assembler Directive & Macro Reference

> The complete syntax reference for Fanticon's native two-pass 6502 assembler — Merlin-inspired source, raw `.BIN` output, project-aware `BANK`/`FIXED` extensions.

Source layout

Source is case-insensitive; string contents are preserved. Merlin-style fields: label at column 1, operation at column 10, operand at column 16, comment at column 27 — whitespace-separated source is also accepted. A semicolon starts a comment outside a quoted string; a line beginning with `*` or `;` is a full-line comment. Labels are global, case-insensitive, and may end in a colon.

Numbers and expressions

Decimal integers, `$`/`0x` hexadecimal, `%` binary, and single-character literals. `*` is the address at the current source line. Unary operators: `+ - ~ <`(low byte) `>`(high byte). Binary operators, highest to lowest precedence: `* /`, then `+ -`, then `<< >>`, then `&`, then `^`, then `|`, and finally comparisons `< > <= >= == !=`. Comparisons produce 1 or 0. Parentheses override precedence. `EQU` and layout-determining directives must resolve when encountered — define those constants first; ordinary instruction/data operands may reference labels defined later.

Addressing modes accepted

Mode	Example
Implied	`RTS`
Accumulator	`ASL A`
Immediate	`LDA #$20`
Zero page or absolute	`LDA ADDRESS`
Indexed	`LDA ADDRESS,X` or `LDX ADDRESS,Y`
Indirect jump	`JMP (VECTOR)`
Indexed indirect	`LDA (POINTER,X)`
Indirect indexed	`LDA (POINTER),Y`

Relative	`BNE LABEL`

Also accepted: Fanticon's stable NMOS undocumented mnemonics `SLO, RLA, SRE, RRA, SAX, LAX, DCP, ISC/ISB, ANC, ALR, ARR, XAA, AXS/SBX, AHX, SHY, SHX, TAS, LAS,` and `KIL/JAM`, plus addressed undocumented `NOP` forms (Appendix A).

Directives

Directive	Purpose
`ORG expr`	Set the address for following output
`name EQU expr / EQ`	Define a constant
`DFB, DB, BYTE`	Emit comma-separated bytes or strings
`DW, DA, WORD`	Emit comma-separated 16-bit values, low byte first
`ASC, TEXT`	Emit quoted ASCII strings or byte expressions
`HEX`	Emit hexadecimal byte pairs, spaces or commas allowed
`DS expr`	Reserve and zero-fill a number of bytes
`PUT, USE, INCLUDE`	Assemble another file at this point (nested up to 16 levels)
`IF/DO, ELSE, ENDIF/FIN`	Select source at assembly time
`REPEAT/LUP, ENDREP/--^`	Repeat source at assembly time
`name PROC, ENDPROC`	Scope dot-prefixed labels to one procedure
`name DUM origin, DEND`	Define a symbol-only data layout
`END`	Mark the logical end of source

Multiple `ORG` regions are allowed only in increasing address order; gaps are zero-filled. Moving backward over already-written output is an error.

Cartridge-project extensions

Directive	Purpose
`BANK 0–255`	Select a switchable 16 KiB image at $8000–$BFFF
`FIXED`	Select the 16 KiB image always mapped at $C000–$FFFF
`BANKOF(label)`	Resolve a switchable-ROM label's bank number, 0–255

Code or data crossing its own section, overlapping earlier output, writing the hidden I/O-shadowed page ($C000–$C0FF within FIXED), selecting an invalid bank, or omitting any CPU vector is a build error. Every unwritten ROM byte is `$FF`.

Modern macros

Define a macro with its name in the label field and `MAC` in the operation field; end with `EOM` or `<<<`. A semicolon- or comma-separated parameter list follows `MAC`. Named parameters use `]NAME`; a trailing parameter may provide a default with `NAME=value`. Positional aliases `]1`, `]2`, and so on remain available, and old definitions with no declared list retain the original `]1` through `]8` behavior:

```
STORE    MAC   VALUE;DEST=$20
         LDA   #]VALUE
         STA   ]DEST
         EOM

         ORG   $8000
         PMC   STORE;$42
         PMC   STORE;$18;$30
```

`PMC name;arg1;arg2` and `>>> name;arg1;arg2` both invoke a macro; a macro name may also appear directly in the operation field. `PMC` is the clearest form for semicolon-separated calls in the editor. Named definitions accept up to 32 parameters. Empty arguments use their defaults. Expansion nests up to 32 levels.

Private labels inside a macro

A label beginning with `@` inside a macro is hygienic: each invocation receives a unique private symbol, and nested expansions remain independent. Reusable loops no longer need caller-supplied label suffixes:

```
WAITX    MAC   COUNT
         LDX   #]COUNT
@LOOP    DEX
         BNE   @LOOP
         EOM

         PMC   WAITX;8
         PMC   WAITX;16
```

Substitution leaves strings and comments unchanged. Missing or extra arguments, invalid parameter declarations, duplicate macro names, unterminated definitions, and unknown calls are reported at their source locations.

Compile-time source control

`IF expression` and Merlin-compatible `DO expression` include a block when the expression is nonzero. `ELSE` selects the other branch; `ENDIF` and `FIN` close it. Conditions may use constants defined by an earlier `EQU`, including comparison expressions:

```
DEBUG    EQU   1
         IF    DEBUG != 0
         LDA   #$E0
         ELSE
         LDA   #0
         ENDIF
```

`REPEAT count;INDEX` and Merlin-compatible `LUP count;INDEX` repeat a block until `ENDREP` or `--^`. `]INDEX` is the zero-based iteration number; `]#` is its short alias. Blocks may nest:

```
         REPEAT 8;PAGE
         STA   $A000+]PAGE*$100,X
         ENDREP
```

Procedure scopes and data layouts

`name PROC` defines a normal entry label and qualifies every dot-prefixed local label until `ENDPROC`. It adds no hidden register saves, stack frame, return, or cycle cost:

```
UPDATE   PROC
.LOOP    DEX
         BNE   .LOOP
         RTS
         ENDPROC
```

The symbols are `UPDATE` and `UPDATE.LOOP`. Another procedure can define its own `.LOOP` without a collision.

`name DUM origin` opens a symbol-only layout. Labeled `DS` fields advance its offset without emitting bytes; `DEND` defines `name.SIZE`. Allocate instances with an ordinary `DS name.SIZE`:

```
PLAYER   DUM   0
X        DS    2
Y        DS    1
FLAGS    DS    1
         DEND

HERO     DS    PLAYER.SIZE
         LDA   HERO+PLAYER.Y
```

Output and limits

Output may span at most the 6502's full 64 KiB address space. Byte and immediate values must fit 8 bits; word and address values must fit 16 bits; branch targets must be in the signed −128..+127 range from the address immediately after the operand — a common workaround for a too-far branch inverts the condition and jumps:

```
         BEQ   NEARBY      ; original intent: BNE FAR_AWAY
         JMP   FAR_AWAY
NEARBY
```

Building

Where	How
Console, raw file	`ASM GAME.ASM [OUTPUT.BIN]`
Console, project	`BUILD` (package), `RUN` (build & launch), `RUN GAME.FCN` (launch existing)
Editor	`F5` / `Ctrl+B` assemble; `Shift+F5` Build & Run

Console diagnostics print as `source:line:column message`. No new binary is ever written on a failed build.

APPENDIX D

Cartridge & Save File Formats

> Both formats are little-endian and independently versioned. Byte-for-byte detail; the narrative version is Chapter 20.

Cartridge capacity

One fixed 16 KiB ROM image, 0–256 switchable 16 KiB ROM banks, and a declaration for 0–4 battery-backed 16 KiB RAM banks. Maximum banked ROM is 4 MiB; the largest possible `.FCN` file, including the fixed image and 64-byte header, is `$404040` bytes (4,210,752). Battery RAM is 0, 16, 32, 48, or 64 KiB, selected with `BANK_KIND=$03`.

.FCN file layout

```
offset $000000  64-byte FCN header
offset $000040  16 KiB fixed ROM image
offset $004040  banked ROM bank 0
offset $008040  banked ROM bank 1
                ...
```

Banks are stored uncompressed in ascending order. The file must be exactly the size implied by its header — trailing bytes are an error.

FCN header (64 bytes)

Off	Size	Type	Field	Meaning
$00	8	bytes	MAGIC	ASCII `FANTICON`
$08	1	u8	FORMAT_MAJOR	$01
$09	1	u8	FORMAT_MINOR	$00
$0A	2	u16	HEADER_SIZE	$0040
$0C	4	u32	FLAGS	Bit 0: BATTERY_RAM
$10	2	u16	ROM_BANKS	Switchable banks, 0–256
$12	1	u8	SAVE_BANKS	Battery RAM banks, 0–4
$13	1	u8	MAPPER	$00 for the v0.1 mapper
$14	4	u32	FIXED_CRC32	CRC-32 of the fixed ROM image
$18	4	u32	BANKED_CRC32	CRC-32 of all banked ROM bytes
$1C	8	u64	CARTRIDGE_ID	Stable, nonzero game identifier

$24	22	bytes	TITLE	Printable ASCII, NUL-padded
$3A	1	u8	MACHINE_MAJOR	Required hardware major version
$3B	1	u8	MACHINE_MINOR	Minimum required minor version
$3C	4	u32	HEADER_CRC32	CRC-32 of header bytes $00–$3B

All CRC fields use CRC-32/ISO-HDLC: polynomial `$04C11DB7`, initial value `$FFFFFFFF`, reflected input/output, final XOR `$FFFFFFFF`. `BANKED_CRC32` is zero when `ROM_BANKS` is zero. Hardware major versions must match exactly; a loader accepts a cartridge only when its own minor version is at least `MACHINE_MINOR`. v0.1 cartridges require machine version 1.0.

Fixed ROM image layout

A complete 16 KiB logical image for `$C000–$FFFF`. Its first 256 bytes correspond to `$C000–$C0FF`, hidden by the I/O page — packagers fill those bytes with `$FF`.

CPU address	Fixed-image offset
$C100	$0100
$FFFA NMI vector	$3FFA
$FFFC RESET vector	$3FFC
$FFFE IRQ/BRK vector	$3FFE

Save file naming and layout

Native: `ASTRO.FCN → ASTRO.SAV`, beside the cartridge, keyed by path. Browser: persistent storage keyed by `CARTRIDGE_ID`.

```
offset $0000  32-byte SAV header
offset $0020  battery-backed RAM bytes
```

Off	Size	Type	Field	Meaning
$00	8	bytes	MAGIC	`FCNSAVE` + $1A
$08	1	u8	FORMAT_MAJOR	$01
$09	1	u8	FORMAT_MINOR	$00
$0A	2	u16	HEADER_SIZE	$0020
$0C	8	u64	CARTRIDGE_ID	Must match the loaded cartridge
$14	4	u32	RAM_SIZE	Exact payload length
$18	4	u32	DATA_CRC32	CRC-32 of RAM payload
$1C	4	u32	HEADER_CRC32	CRC-32 of header bytes $00–$1B

`RAM_SIZE` must be 16, 32, 48, or 64 KiB, and the file must be exactly `HEADER_SIZE + RAM_SIZE` bytes. If a valid save's `RAM_SIZE` does not match `SAVE_BANKS × $4000` for the loading cartridge, Fanticon deletes it and creates a fresh zero-filled save at the correct size.

Mapper 0 (the only mapper in format 1.0)

BANK_KIND	Meaning	Bank range
$00	Cartridge ROM	0 to ROM_BANKS−1
$01	Work RAM	0–3
$02	Video RAM	0–2
$03	Battery-backed save RAM	0 to SAVE_BANKS−1

Loader validation order

1. Minimum length and magic
2. Supported major/minor version and header size
3. Known flags and mapper
4. ROM and save bank-count limits
5. Nonzero cartridge ID, valid title bytes, compatible machine version
6. Exact total file size
7. Header, fixed-ROM, and banked-ROM CRC values
8. Fixed-ROM vector presence

This format stores no host paths, no executable host code, no compression stream, and no embedded save data.

APPENDIX E

Demo Cartridge Index

> Every example cartridge ships under `/demos` the moment the host boots. Each is small enough to read end to end in one sitting.

Folder	Demonstrates	See
`graphics`	The complete .GFX/.PAL asset pipeline: edit patterns, a 64×32 map, attributes, and 16×16 sprite art, then load all of it into VRAM at runtime	Ch. 8
`tiles`	A full tilemap scroller driven by a VBlank IRQ reading controller 1; scroll registers wrap around a 320×200 map	Ch. 9
`sprites`	One hardware sprite, VBlank-driven movement, edge clipping, and signed coordinates	Ch. 10
`bitmap`	Filling the packed 320×200 bitmap across VRAM banks 1–2; visible multi-frame startup cost	Ch. 11
`raster`	Four horizontal color bands from raster IRQs alone, with zero framebuffer writes	Ch. 12
`wave`	A per-scanline raster IRQ bending tilemap scroll along a 128-step sine curve, timed against HBlank	Ch. 12
`audio`	All four voices together, with a VBlank IRQ changing the lead note and backdrop color in lockstep	Ch. 14
`music`	A complete tracker song ("Ode to Joy") plus `PLAYER.INC`, a reference cartridge music player	Ch. 15

Every demo folder also carries its own `FANTICON.CFG` and `readme.txt` — from any of them, `RUN` or `F5` builds and launches it on the spot.

APPENDIX F

Glossary

Term	Meaning
Bank	One 16 KiB slice of ROM, work RAM, VRAM, or save RAM, selectable into the $8000–$BFFF window
BCD	Binary-coded decimal; the NMOS decimal-mode arithmetic adjustment
Cartridge	A validated .FCN image: fixed ROM, banked ROM, and metadata
Fantasy console	An emulated machine with no historical hardware counterpart, built and documented as if it had one
Fixed ROM	The 16 KiB image always mapped at $C000–$FFFF, holding RESET and the CPU vectors
HBlank	Horizontal blanking; dots 320–399 of each scanline
IRQ	Maskable interrupt request; shared and software-prioritized among four sources
LFSR	Linear-feedback shift register; drives the noise channel's pseudo-random output
NMI	Non-maskable interrupt; reserved by v0.1, undriven by standard devices
NMOS 6502	The original MOS Technology 6502 variant Fanticon emulates, including undocumented opcodes, as distinct from the later CMOS 65C02
Page crossing	An indexed address whose high byte differs from its unindexed base — costs an extra CPU cycle on affected instructions
RGB332	An 8-bit color encoding: 3 bits red, 3 bits green, 2 bits blue
Raster	The simulated scanning beam that paints the display top to bottom, left to right, whose exact position code can read and race
VBlank	Vertical blanking; scanlines 200–261, when nothing visible is being drawn
VRAM	Video RAM; 48 KiB across three banks, holding patterns, maps, attributes, sprite records, or bitmap pixels
W1C	Write-one-to-clear; the acknowledgment pattern used by IRQ_PENDING
Zero page	Addresses $0000–$00FF, with shorter and faster addressing modes than the rest of memory